

Safe Conservative GC is What You Need

Anonymous

Abstract

Conservative garbage collection scans the full thread state—stack and registers—for values that resemble pointers, avoiding the bookkeeping that precise rooting demands. It has been the workhorse of legacy runtimes since Boehm’s 1993 paper, because it trades a little sloppiness for a great deal of simplicity. The sloppiness has a price the community has long accepted: some values in the thread state are not pointers at all—old integers, leftover floats, fragments of addresses—yet the collector cannot tell them apart from genuine references. These *false roots* pin garbage in place. A particularly stubborn kind can reappear every few collection cycles, each time renewing the object’s lease on life. For thirty years, the bound on how long a false root may survive has been, in effect, “forever.”

We argue that the bound can be made finite. A false root is not a permanent fact about the world; it is a stale observation. The stack is transient—a value appears in a slot, then the slot is overwritten. What makes a false root dangerous is not its existence but its repeated reappearances, each resetting the object’s color from white back to gray. If the collector simply remembered that it had seen this particular suspect before, and refused to reset the clock on later sightings, the renewal channel would close.

We formalize this as *False-Root Aging Theory*. Two mechanisms do the work, and they are independent. A *lease* records the epoch of a candidate root’s first appearance and refuses to refresh that timestamp on subsequent sightings, so an oscillating false root cannot renew itself indefinitely. A *punitive age* counts how many cycles an object has survived on conservative evidence alone—never confirmed by a real heap trace—and forces reclamation once the count crosses a threshold. Objects that the heap trace actually reaches are exempt: their age freezes at zero and stays there. The two mechanisms are like a statute of limitations and a strike count: one bounds survival by time, the other by accumulated suspicion, and either suffices on its own.

The theory has two orthogonal instantiations. The extrinsic lease stores its state in a small external table keyed by address, modifying no object header; it is portable to legacy runtimes such as CPython or the Boehm collector, at the cost of an address-reuse hazard. The intrinsic lease stores its state in the object header, eliminating the hazard at the cost of one bit. Both give a bounded lifetime: $\Theta + 1$ epochs for the intrinsic lease, $\mathcal{L} + 1$ for the extrinsic. No trace-reachable object is ever reclaimed.

We implemented the intrinsic lease in a real virtual machine. False roots vanish in exactly the predicted number of collection cycles: $\Theta + 1 = 4$ for transient roots under the base theory, $P \cdot (\Theta + 1) + 2 = 10$ for oscillating ($P = 2$) roots under DRD. Stop-the-world pauses stay under $25 \mu\text{s}$ regardless of heap size. The predictions are confirmed by three experiment groups: a controlled convergence test, performance characterization, and a real-workload allocation pattern plus a 4-thread multi-mutator stress test that exercises the multi-mutator safety and convergence theorems with zero UAF.

The theory’s safety rests on axiom 4 (state completeness), which assumes the conservative scan observes the full thread state $\Sigma = \text{Stack} \cup \text{Registers}$ at every safepoint. Real runtimes are reliable but not provably perfect: a JIT may transiently hold a pointer in a register not captured by the signal context, or a concurrent marker may race on a field for one cycle. We address this with *Defensive Re-verification Demotion* (DRD), a two-safepoint consensus that replaces the single-shot force-demotion with a defer-then-confirm protocol. DRD does not eliminate all use-after-free risk—a pointer persistently invisible to every scan cannot be protected by any conservative mechanism—but it converts the single-shot vulnerability into a two-safepoint consensus, reducing the risk quadratically for the transient case. The cost is a +2-cycle extension of the oscillating bound.

The one case the intrinsic age cannot bound is the persistent false root ($P = 1$), where Age freezes at 0 and $L = \infty$. We close this boundary with a *hybrid lease* that combines the intrinsic age with an extrinsic, address-keyed, wall-clock lease. The extrinsic lease expires after $\mathcal{L}$ epochs regardless of whether the candidate reappears, so the persistent false root becomes White once the lease expires and is reclaimed by the EBR delay. The hybrid bound is $L \leq \mathcal{L} + 2$, and no false-root category remains unbounded.

1 Introduction

A garbage collector’s job is a bookkeeping problem at heart. The collector must find every object the program still needs and reclaim the rest. It starts from the roots—the variables the program can reach right now—and walks the pointer graph, marking everything it encounters. Whatever it did not reach is garbage.

Finding the roots should be the easy part. A precise collector gets a list from the compiler: “here are the registers and stack slots that hold pointers.” The collector trusts the list. But many runtimes never got the compiler’s cooperation. CPython, Lua, older Smalltalks, the Boehm–Demers–Weiser collector for C and C++—all scan the native stack, word by word, and ask of each value a simple question: “does this look like it points into the heap?” If yes, treat it as a root. This is *conservative collection*. It is sloppy by design, and it has served legacy runtimes for three decades.

1.1 The Conservative Wound

The sloppiness has a price. Some stack values are not pointers. They are old integers, leftover floating-point bits, fragments of addresses from a previous call frame. But they happen to land inside the heap’s address range, so the collector treats them as roots. These are *false roots*, and they pin garbage. An object that should have been reclaimed stays alive because a stale integer on the stack points at it.

A single false root is usually tolerable—a little memory waste, no correctness issue. The real trouble begins when the false root *oscillates*. A loop reuses a stack slot; the same bogus pointer value reappears every few iterations; each reappearance re-marks the object gray, pulling it back from the brink of reclamation. The object never dies. Its lifetime is unbounded, and the bound depends on the program’s control flow, not on any collector parameter.

Boehm recognized this in 1993 and offered a partial fix: *blacklist* addresses that produced false roots and avoid re-scanning them [1]. The blacklist is reactive—an address must first cause harm to be blacklisted—and it offers no bound on how long a persistent false root may survive. SBCL erases uninitialized stack slots defensively, but a live computation can still leave a stale value behind. Shadow stacks [3] and precise rooting [4] eliminate false roots entirely but require compiler cooperation that many runtimes cannot afford. Generational collectors [5] use age to *reward* survivors, which is the opposite of what safety demands.

The consensus, stated plainly: safe collection requires precise roots; conservative collection is a convenience that cannot be made safe.

1.2 The Temporal Dimension

The consensus above is too pessimistic, and the reason is simple: a false root is not a permanent fixture. It is a stale observation. The stack is transient. A value shows up in a slot, then the slot is overwritten. The danger is not that a false root exists, but that it can reappear—a loop reuses a stack slot, an integer constant cycles through a register, and the same bogus pointer resurfaces epoch after epoch, each time renewing the object’s survival authority.

The infinite renewal, not the false root itself, is the source of unboundedness. Sever the renewal channel, and the false root becomes an ordinary transient: it appears, it fails to get confirmed by the heap trace, and it eventually fades.

This suggests a different cure. We need not eliminate false roots. We need only remember that we have seen them before, and stop taking their word for it.

1.3 Contributions

We present *False-Root Aging Theory*, a formal framework that gives conservative collection a mathematical safety bound. The theory rests on two independent mechanisms, each addressing a distinct failure mode:

- **The lease** records when a candidate root first appeared and refuses to reset that clock on later sightings. An oscillating false root cannot renew itself indefinitely; after a bounded term, the candidate is permanently ignored. This severs the renewal channel.

- **The punitive age** counts how many cycles an object has survived on conservative evidence alone—never confirmed by a real heap trace—and forces reclamation once the count crosses a threshold. True roots, reached by the object graph, are exempt: their age freezes at zero.
- **Two orthogonal instantiations.** The extrinsic lease stores its state in a small external table keyed by address, modifying no object header. It is portable to legacy runtimes, at the cost of an ABA hazard when addresses are reused. The intrinsic lease stores its state in the object header, eliminating the ABA hazard at the cost of one bit. Both yield a bounded convergence theorem with different constants.
- **Formal proofs.** We prove that any false root’s lifetime is bounded (Theorem 2), and that no trace-reachable object is ever reclaimed (Theorem 3). Under the intrinsic lease, the bound is $\Theta + 1$ epochs, where Θ is the age threshold.
- **Empirical validation.** We implemented the intrinsic lease in a real virtual machine. False roots converge to zero survivors in exactly the predicted number of cycles: $\Theta + 1 = 4$ for transient roots under the base theory, $P \cdot (\Theta + 1) + 2 = 10$ for oscillating ($P = 2$) roots under DRD. Stop-the-world pauses stay under $25 \mu\text{s}$ regardless of heap size.

Conservative collection need not be a convenience trade-off. It can have a bound.

2 Preliminaries

Before we can state the problem precisely, we need a small amount of notation. Most of it is standard tracing-collector terminology; the only nonstandard piece is the epoch clock, which gives us a way to talk about “how long ago” something happened.

2.1 Tracing Collection

Let $\mathcal{O}$ be the set of heap objects and $\mathcal{R}$ the root set. Define the reachability relation $\xrightarrow{*}$ as the reflexive-transitive closure of the child-pointer relation. An object $o \in \mathcal{O}$ is *live* if there exists $r \in \mathcal{R}$ with $r \xrightarrow{*} o$, and *dead* otherwise. A tracing collector partitions $\mathcal{O}$ into live and dead by computing this closure, then reclaims the dead.

The standard algorithm is mark-sweep: a mark phase colors reachable objects, a sweep phase reclaims the uncolored remainder. The correctness condition is straightforward—every dead object is eventually reclaimed, and no live object is ever reclaimed.

2.2 Tri-Color Marking

The mark phase uses three colors, assigned per object per collection cycle:

- **White:** not yet visited. Candidates for reclamation.
- **Gray:** visited, but whose children have not been fully traced.
- **Black:** visited, and whose children have been fully traced.

A gray worklist drives the traversal: a white object reached from a gray object is colored gray and added to the worklist; a gray object whose children are exhausted is colored black. When the worklist empties, marking is complete, and any remaining white object is unreachable.

2.3 Conservative Root Sets

In a precise collector, $\mathcal{R}$ is computed exactly: each root is a known pointer. In a conservative collector, the thread’s full computational state is scanned for values that *could* be pointers, and every such value is treated as a root. The computational state Σ of a paused thread comprises both the stack and the register file:

$$\Sigma = \text{Stack} \cup \text{Registers}$$

The conservative root set is:

$$\mathcal{R} = \mathcal{R}_{\text{exact}} \cup \mathcal{R}_{\text{cons}}$$

where $\mathcal{R}_{\text{exact}}$ are explicitly registered roots (known pointers) and $\mathcal{R}_{\text{cons}}$ are values discovered by scanning Σ that happen to fall within the heap address range.

At any safepoint, the collector has complete observability of Σ : the operating system’s context-switch mechanism (or equivalent signal handling) atomically captures the register file into a memory buffer, which the collector scans word by word alongside the stack. This is not an implementation convenience but a theoretical invariant: a value resident in a register at the safepoint is physically present in the hardware and is observable by the collector. It does not “disappear” because the mutator did not spill it; it is part of Σ .

The fundamental difficulty: deciding whether a value in Σ is a genuine pointer or just an integer that looks like one is undecidable in general—a consequence of Rice’s theorem applied to the program’s semantics. So $\mathcal{R}_{\text{cons}}$ is an over-approximation. It contains true roots, but it may also contain *false roots*: values that are not pointers but are mistaken for them. A false root $r \in \mathcal{R}_{\text{cons}}$ can pin an otherwise dead object o if $r \xrightarrow{*} o$, even though no genuine reference path exists.

2.4 Epoch-Based Reclamation

We assume a global monotonically increasing clock Φ_t , the *epoch sequence*, where $t \in \mathbb{N}$ indexes collection cycles. Each object o carries a timestamp $\Phi_{\text{touch}}(o)$ recording the epoch of its last allocation or mutation. Reclamation is deferred by one epoch to ensure no in-flight mutator holds a stale reference:

$$\text{Reclaimable}(o) \implies \Phi_t - \Phi_{\text{touch}}(o) > 1$$

This is the standard epoch-based reclamation (EBR) invariant. We adopt it as a temporal axiom throughout: an object can be reclaimed only after its timestamp is at least two epochs behind the current one. The EBR delay is what gives our theory its unit of time—when we say a false root “survives k epochs,” we mean k collection cycles, each of which advances Φ by one.

2.5 Limitations of Existing Approaches

Several strategies mitigate false-root retention, but none bounds the lifetime of an individual false root:

Boehm blacklisting [1] passively records addresses that produced false roots and avoids re-scanning them. It is reactive: a root must first cause harm to be blacklisted, and the blacklist offers no guarantee on how long a persistent false root survives.

Stack slot erasure (used by SBCL) zeroes uninitialized stack frames defensively. It relies on compiler cooperation and cannot prevent a live computation from leaving a stale value in a slot the erasure pass did not anticipate.

Shadow stacks [3] and **precise rooting** [4] eliminate $\mathcal{R}_{\text{cons}}$ entirely by requiring the compiler to maintain an explicit root set. They solve the problem by removing it, but the cost—compiler instrumentation, ABI changes—is too high for many legacy runtimes.

Generational age [5] counts how many collections an object has survived and promotes long-lived objects to an older generation. The semantics are the opposite of what false-root safety requires: generational age rewards survivors; we need a mechanism that punishes suspects.

None of the above provides a per-false-root lifetime bound. The rest of the paper builds one.

3 Problem: Oscillating False Roots

With the notation in place, we can now state precisely why pure color guarding fails. The failure is not subtle—it is a structural blindness to time.

3.1 Pure Color Guarding

A tracing collector that decides liveness solely by the current cycle’s color implements what we call *pure color guarding*:

$$\text{Alive}_t(o) \iff \text{Marked}_t(o) \vee \text{ReachableFromHeap}_t(o)$$

Here $\text{Marked}_t(o)$ means o was colored gray or black during cycle t 's mark phase, and $\text{ReachableFromHeap}_t(o)$ means o is reachable from a marked object. The sweep phase reclaims exactly those objects for which $\text{Alive}_t(o)$ is false, subject to the EBR delay.

Pure color guarding has no memory. It asks one question—"was o marked this cycle?"—and never asks "how many times has o been marked across cycles without ever being confirmed by the heap trace?" This is the gap we will close.

3.2 Oscillating False Roots

Definition 1 (Oscillating false root). *A false root $r \in \mathcal{R}_{\text{cons}}$ is oscillating if there exists an infinite strictly increasing sequence of epochs $\{t_i\}_{i=0}^\infty$ such that:*

- *at each t_i , the stack slot containing r holds a value pointing to some object o ,*
- *at epochs strictly between t_i and t_{i+1} , the slot does not point to o ,*
- *o is never reached by the heap trace ($\forall t, \neg \text{ReachableFromHeap}_t(o)$).*

A special case worth naming is the *periodic false root*, where $t_{i+1} - t_i = P$ for a constant period P . This happens when a loop body reuses a stack slot every P cycles: the same bogus pointer value recurs reliably. Periodic false roots are the practically dangerous case, because they recur indefinitely and are invisible to any single-cycle inspection.

3.3 Unbounded Survival

Theorem 1 (Unbounded survival). *Under pure color guarding, an oscillating false root r that pins an object o (as in definition 1) gives o an unbounded lifetime:*

$$L(o) = \infty$$

where $L(o)$ is the number of collection cycles from o 's allocation to its reclamation.

Proof. At each epoch t_i in the oscillation sequence, $\text{Marked}_{t_i}(o)$ is true: $r \in \mathcal{R}_{\text{cons}}$ is treated as a root, and the mark phase colors o gray. The sweep at t_i therefore skips o . Between t_i and t_{i+1} , o may transition to white (no mark), but it is not reclaimed because either the EBR delay $\Phi_t - \Phi_{\text{touch}}(o) > 1$ does not yet hold, or the next oscillation at t_{i+1} re-marks o before the delay can elapse. Since $\{t_i\}$ is infinite and o is never confirmed unreachable—it is perpetually re-marked— o is never in a state where $\text{Reclaimable}(o)$ holds persistently. Thus o survives forever. $\square$

Corollary 1. *Any mechanism that bounds $L(o)$ for oscillating false roots must sever the "reappearance $\Rightarrow$ renewal" channel. The fact that r reappears at t_{i+1} must not reset o 's survival authority to the same level as a first-time sighting.*

Corollary 1 is the design imperative for the rest of the paper. Pure color guarding fails because it is memoryless: every cycle is a fresh verdict, indistinguishable from every other cycle. The theory we develop next introduces memory—in two complementary forms.

4 False-Root Aging Theory

This section builds the formal framework. We state the aging axioms (section 4.1), then the two mechanisms that implement them: the lease (section 4.2), which severs the renewal channel, and the punitive age (section 4.3), which accumulates suspicion. The true-root exemption (section 4.4) gives the safety guarantee, and the convergence theorem (section 4.5) ties the pieces together. A robustness extension, DRD, appears at the end (section 4.6).

4.1 Aging Axioms

The theory rests on three axioms. They specify what information must exist and how it must influence decisions, but say nothing about where the bits live—that is a matter for the two instantiations in sections 4.2 and 4.3.

The first axiom is the one that makes the whole theory possible. The collector must remember *why* an object was marked: because the heap trace reached it (a strong signal), or because the stack scan found a value pointing at it (a weak signal)? Pure color guarding collapses these two cases into one bit—marked or not—and that collapse is precisely what makes oscillating false roots invisible.

Axiom 1 (Source distinction). *The source of a mark—trace-confirmed ($\mathcal{T}$) versus conservative-speculative ($\mathcal{C}$)—must be stored orthogonally to the mark state (white, gray, black). The source is the input to every aging decision.*

The second axiom says that an object whose survival rests solely on conservative evidence should not enjoy the same confidence as a trace-confirmed object. Its confidence must decay. This is the temporal half of the theory: pure color guarding is memoryless across cycles, and memorylessness is what theorem 1 exploits.

Axiom 2 (Temporal decay). *An object whose survival rests solely on conservative evidence (source = $\mathcal{C}$) has a survival confidence that decreases monotonically across collection cycles. It does not remain constant.*

The third axiom is the safety guarantee. Trace confirmation grants exemption for the cycle in which the trace reaches the object; the exemption must be renewed by re-tracing each cycle. This is what prevents the theory from reclaiming a live object, while still allowing objects that become garbage to be reclaimed normally.

Axiom 3 (True-root exemption, per-cycle). *If the heap trace reaches an object o in cycle t (source becomes $\mathcal{T}_t(o)$), then within cycle t : its lease is cleared, its age is reset to zero, and it is exempt from reclamation for that cycle. The exemption is renewed each cycle by re-tracing: if o is not reached in cycle $t + 1$, its source reverts to $\mathcal{C}$ (or $\perp$), and it is subject to normal aging and reclamation.*

The fourth axiom establishes that the conservative scan has complete observability of the thread’s computational state. It ensures that no false root can escape the aging mechanisms by residing in a register rather than on the stack.

Axiom 4 (State completeness). *At any safepoint, the conservative scan observes the full thread state $\Sigma = \text{Stack} \cup \text{Registers}$. No value in Σ is invisible to the scan. A value resident in a register at the safepoint is treated identically to a value on the stack: it is a candidate root, subject to the lease and age mechanisms.*

This axiom eliminates the need for any special-case treatment of registers. In practice, the operating system’s context-switch mechanism (or signal handling) atomically captures the register file into a memory buffer at the safepoint, and the collector scans this buffer alongside the stack. The aging theory operates on whatever candidates the scan produces, regardless of whether a candidate originated from the stack or the register file. The lease anchors, the age accumulates, and the forced demotion fires—all uniformly.

These four axioms are implementation-agnostic. The remaining subsections give them concrete operational rules and prove they yield a bounded lifetime.

4.2 First Defense: The Lease

The lease is our answer to corollary 1: it severs the renewal channel by remembering when a candidate root first appeared and refusing to refresh that memory on later sightings. An oscillating false root can reappear as many times as it likes; the lease will not be fooled into treating each reappearance as a fresh first sighting.

4.2.1 Lease State and Rules

Definition 2 (Lease state). *Each candidate root r (an address or an object identity, depending on instantiation) is associated with a lease state:*

$$\text{Lease}(r) \in \{\text{ACTIVE}, \text{EXPIRED}, \perp\}$$

where $\perp$ denotes “no lease recorded.” If $\text{Lease}(r) = \text{ACTIVE}$, a first-seen epoch $\Phi_{\text{first}}(r)$ is recorded alongside it.

Rule 1 (First anchoring). When r first appears as a candidate root, set $\text{Lease}(r) := \text{ACTIVE}$ and record $\Phi_{\text{first}}(r) := \Phi_{\text{current}}$.

Rule 2 (No renewal). When r reappears, $\Phi_{\text{first}}(r)$ is not updated. The timer is not reset.

Rule 3 (Lease expiration). If $\Phi_{\text{current}} - \Phi_{\text{first}}(r) > \mathcal{L}$, then $\text{Lease}(r) := \text{EXPIRED}$, and the candidate root is permanently ignored: it is not passed to the marker.

Rule 4 (True-root exemption). If the object that r points to is ever reached by the heap trace (source becomes $\mathcal{T}$), then $\text{Lease}(r) := \perp$ unconditionally. The candidate is no longer subject to any lease.

?? 1?? 2 are the heart of the mechanism. Rule 1 records the first sighting; rule 2 ensures that subsequent sightings do not reset the clock. Together they implement the *non-renewability* that corollary 1 demands. An oscillating false root can keep reappearing, but its lease was anchored at t_0 , and that anchor never moves.

4.2.2 Extrinsic Lease

The *extrinsic lease* instantiates the lease as an external address-indexed table, modifying no object header.

Definition 3 (Extrinsic lease table). The *extrinsic lease* is a partial map from addresses to first-seen epochs:

$$\Lambda_E : \mathcal{A} \rightarrow \mathbb{N} \cup \{\perp\}$$

where $\mathcal{A}$ is the address space. The table has fixed capacity C (a constant independent of heap size). For an address a :

$$\Lambda_E(a) = \begin{cases} t & \text{if } a \text{ first appeared as a candidate root at epoch } t \\ \perp & \text{if } a \text{ never appeared, or its lease was cleared} \end{cases}$$

A candidate root a is passed to the marker if and only if:

$$\Lambda_E(a) = t \quad \wedge \quad \Phi_{\text{current}} - t \leq \mathcal{L}$$

Proposition 1 (Capacity bound). The number of valid candidate roots in any single collection cycle is small (empirically < 3) and is independent of stack depth. A constant capacity $C = 16$ is a safe upper bound.

Sketch. A candidate root is a stack value that points into the heap and resolves to a known object. The number of such values is bounded by the number of live references the mutator holds on the stack at the safepoint, which is a property of the program, not the stack’s address space. Empirically, even deeply recursive programs rarely hold more than a handful of heap pointers per frame, and the total across a typical call depth is well below 16. $\square$

Advantage. The extrinsic lease requires no modification to any object header. It can be deployed on legacy runtimes (CPython, Boehm GC) by inserting a lookup step in the existing stack scan routine.

Limitation. The lease is keyed by address, not object identity. If an object at address a is reclaimed and a is later reused for a new object o' , the stale lease entry (a, t) may incorrectly expire o' ’s root before o' has had a fair trial. This is the *ABA hazard*: address reuse breaks the lease’s invariant. The intrinsic lease (section 4.2.3) eliminates this hazard.

4.2.3 Intrinsic Lease

The *intrinsic lease* instantiates the lease as a per-object attribute, binding it to object identity rather than address.

Definition 4 (Intrinsic lease state). Each object o carries a lease flag $\text{conservative}(o) \in \{0, 1\}$ and a mark epoch $\Phi_{\text{mark}}(o)$. The flag records whether o ’s current mark was set by the conservative scan (1) or by the heap trace (0). The epoch records when the mark was set.

The mark operation splits into two variants:

- **Trace mark** (applied when the heap trace reaches o): sets $\Phi_{\text{mark}}(o) := \Phi_{\text{current}}$ and $\text{conservative}(o) := 0$.
- **Conservative mark** (applied when the stack scan discovers o as a candidate): sets $\Phi_{\text{mark}}(o) := \Phi_{\text{current}}$ and $\text{conservative}(o) := 1$.

If o was first marked conservatively and later reached by the trace, the trace mark overwrites the conservative mark, clearing $\text{conservative}(o)$. This is the operational form of axiom 3.

Advantage. The lease is bound to the object, not the address. When o is reclaimed, its lease state is destroyed with it. A new object o' allocated at the same address starts with a fresh lease state ($\text{conservative} = 0$). The ABA hazard is eliminated.

Cost. One bit of object-header storage for conservative, plus storage for the age counter (section 4.3).

4.3 Second Defense: Punitive Age

The lease severs the renewal channel, but it does not, by itself, force reclamation. A candidate root whose lease has not yet expired still gets passed to the marker, and the object it points to still gets marked gray. Something has to count how many times this has happened and eventually say “enough.”

That something is the punitive age. It is a per-object counter that ticks upward each time the object is marked on conservative evidence alone, and it triggers forced reclamation when it exceeds a threshold. The age is the theory’s other half, and it is independent of the lease: the lease decides *whether* a candidate reaches the marker, the age decides *how long* a reached object may survive without confirmation.

4.3.1 Source State Predicate

Definition 5 (Source state). *Each object o has a source predicate $\mathcal{S}(o) \in \{\mathcal{T}, \mathcal{C}\}$:*

- $\mathcal{T}$ (trace-confirmed): o has been reached by the heap trace at least once.
- $\mathcal{C}$ (conservative-speculative): o has never been reached by the trace; its survival rests solely on conservative roots.

Rule 5 (State irreversibility). *The transition $\mathcal{S}(o) = \mathcal{C} \Rightarrow \mathcal{S}(o) = \mathcal{T}$ is irreversible. Once an object is confirmed by the trace, it never returns to $\mathcal{C}$.*

Irreversibility is what makes the theory safe within a cycle. An object that the heap graph has vouched for in cycle t is vouched for that cycle; no conservative noise in cycle t can undo that. Across cycles, the lazy epoch resets the effective state: if the object is not re-traced in cycle $t + 1$, it is White, regardless of its $\mathcal{T}$ status in cycle t .

4.3.2 Age Counter

Definition 6 (Age counter). *Each object o has a non-negative integer $\text{Age}(o)$, initialized to 0. At each collection cycle t :*

$$\text{Age}_{t+1}(o) = \begin{cases} \text{Age}_t(o) + 1 & \text{if } \neg \text{Marked}_t(o) \wedge \text{strong}(o) = 0 \\ 0 & \text{if } \mathcal{S}(o) = \mathcal{T} \\ \text{Age}_t(o) & \text{otherwise} \end{cases}$$

The age increments only when o is *not marked* this cycle (the conservative scan did not find a value pointing to it) and its strong count is zero. When o is marked (by either the conservative scan or the heap trace), the age *freezes*—it neither increments nor resets. This is the critical distinction: freezing (not resetting) preserves the accumulated suspicion from prior disappearance cycles, so that an oscillating false root’s age monotonically increases across its disappearance cycles, even though it is frozen during its appearance cycles.

This formulation preserves both safety and convergence:

- **True roots are protected:** if a local variable on the stack continuously points to o , the conservative scan marks o every cycle, $\text{Age}(o)$ freezes at its initial value 0, and o is never reclaimed.

- **Oscillating false roots converge:** if the stack value pointing to o disappears (the slot is overwritten), o is not marked, $\text{Age}(o)$ increments. When the value reappears, $\text{Age}(o)$ freezes—it does not reset. The age monotonically increases across disappearance cycles, and after $\Theta + 1$ cumulative disappearances, forced demotion fires.
- **Persistent false roots are a known boundary:** if a false root appears every cycle (never disappears from the stack), $\text{Age}(o)$ freezes at 0 permanently, and o is never reclaimed. This is indistinguishable from a true root in pure conservative scanning—the boundary stated in theorem 2. (The hybrid lease in section 4.7 closes this boundary.)
- **Trace-confirmed objects are exempt for the cycle:** $\mathcal{S}(o) = \mathcal{T}$ in cycle t resets $\text{Age}(o)$ to 0 for that cycle. The exemption is renewed by re-tracing; if the object is not re-traced in cycle $t + 1$, it becomes White and is reclaimed normally.

Note the asymmetry with generational age. Generational age counts *survivals* and rewards the object with promotion. Punitive age counts *disappearances* (failed markings) and punishes the object with eventual reclamation. The two mechanisms are mathematical opposites; they can coexist in the same collector, but they serve different masters.

4.3.3 Forced Demotion

Definition 7 (Forced demotion). *When $\text{Age}(o) > \Theta$ (where Θ is the age threshold), the operation $\text{Demote}(o)$ is applied:*

$$\text{Demote}(o) : \Phi_{\text{mark}}(o) \leftarrow 0$$

This resets the mark epoch to 0, making o appear as “prehistoric garbage” in the EBR sense: the object is now in a reclaimable state, subject to the standard EBR delay.

Forced demotion is the theory’s escape hatch. It is, to our knowledge, the first operation in EBR-based collection that allows an individual object’s timestamp to be reset. The reset is safe because it only applies to objects whose source is $\mathcal{C}$ —never to trace-confirmed objects—and because the EBR delay still applies: the object is not reclaimed in the same cycle it is demoted, only in a later sweep where the delay has elapsed.

4.3.4 Contrast with Generational Age

Dimension	Generational age	Punitive age
Semantics	Reward survivors	Punish suspects
Threshold action	Promote to old generation	Force reclaim
Math direction	Positive feedback	Negative feedback
Design goal	Optimize throughput	Guarantee convergence

Table 1: Generational age versus punitive age.

4.4 True-Root Exemption

We now state the safety guarantee operationally. Axiom 3 says that trace-confirmed objects are exempt; here we spell out what “exempt” means for each mechanism:

If $\mathcal{S}(o) = \mathcal{T}$ in cycle t , then within cycle t :

- $\text{Lease}(o) := \perp$ (the lease is cleared for this cycle),
- $\text{Age}(o) := 0$ (the age is reset to zero this cycle),
- $\text{Reclaimable}(o) = \text{False}$ (the object is not a reclamation candidate this cycle).

Formally:

$$\mathcal{S}_t(o) = \mathcal{T} \implies \text{Age}_t(o) = 0 \wedge \text{Lease}_t(o) = \perp \wedge \neg \text{Reclaimable}_t(o)$$

The exemption is *per-cycle, renewed by re-tracing*. The trace must reach o in cycle t to set $\mathcal{S}_t(o) = \mathcal{T}$. If o is not reached in cycle $t + 1$ (because it became garbage), then $\mathcal{S}_{t+1}(o) \neq \mathcal{T}$ —the object is White, its age accumulates, and it is reclaimed after the EBR delay. This is the operational form that prevents the “permanent exemption leak”: a one-time trace confirmation does not grant lifetime immunity; it grants immunity for the cycle in which the trace reached the object, and the immunity must be renewed each cycle by re-tracing. An object that was live in cycle t and dead in cycle $t + 1$ is reclaimed normally—the lazy epoch mechanism (definition 6) makes the prior $\mathcal{T}$ label stale when $\Phi_{\text{mark}}(o) < \Phi_{\text{current}}$.

The source predicate $\mathcal{S}(o)$ is therefore best understood as $\mathcal{S}_t(o)$: a per-cycle attribute, not a permanent one. Rule 5 (state irreversibility) holds within a cycle: once $\mathcal{T}$ is set in cycle t , no conservative noise in cycle t can unset it. But across cycles, the lazy epoch resets the effective state: an object not re-traced in cycle $t + 1$ is White, regardless of its $\mathcal{T}$ status in cycle t .

4.5 Unified Convergence Theorem

We now tie the pieces together. The lease and the punitive age are independent mechanisms, but they cooperate: the lease decides which candidates reach the marker, the age decides how long a reached object may survive without confirmation. Together they bound the lifetime of any false root.

4.5.1 Convergence Path

For any false root o ($\forall t, \mathcal{S}_t(o) = \mathcal{C}$), the following sequence is forced:

1. **Lease anchors.** At first appearance (epoch t_0), the lease records $\Phi_{\text{first}} = t_0$. Subsequent appearances do not refresh.
2. **Age accumulates on disappearance.** Each cycle that o is *not* marked (the conservative scan did not find a value pointing to it—the stack slot has been overwritten) increments $\text{Age}(o)$. Cycles where o is marked freeze $\text{Age}(o)$ at its current value.
3. **Threshold triggers demotion.** When $\text{Age}(o) > \Theta$ (after $\Theta + 1$ cumulative disappearance cycles), $\text{Demote}(o)$ resets $\Phi_{\text{mark}}(o) \leftarrow 0$.
4. **Reclamation.** In the next sweep, o is white (stale mark) and the EBR delay is satisfied. The object is reclaimed.

4.5.2 Bounded Convergence

Theorem 2 (Bounded false-root convergence). *Let o be an object with $\forall t, \mathcal{S}_t(o) = \mathcal{C}$. Under the aging mechanisms:*

- Under the *extrinsic lease*: $L(o) \leq \mathcal{L} + 1$, where $\mathcal{L}$ is the lease term.
- Under the *intrinsic lease*, for a *transient* false root (appears once at t_0 , then never reappears): $L(o) \leq \Theta + 2$.
- Under the *intrinsic lease*, for an *oscillating* false root with period $P \geq 2$ (reappears every P cycles): $L(o) \leq P \cdot (\Theta + 1)$.
- Under the *intrinsic lease*, for a *persistent* false root ($P = 1$, appears every cycle): $L(o) = \infty$.

Proof (intrinsic lease, transient). Let t_0 be the epoch of o ’s first (and only) appearance. At t_0 , o is marked, and $\text{Age}(o) = 0$ (frozen).

For each cycle $t_0 + k$ ($k = 1, 2, \dots$), o is not marked (the false root has disappeared). By definition 6, $\text{Age}(o)$ increments: $\text{Age}_{t_0+k}(o) = k$.

At $k = \Theta + 1$, $\text{Age}(o) = \Theta + 1 > \Theta$, so $\text{Demote}(o)$ fires, setting $\Phi_{\text{mark}}(o) := 0$.

At the sweep of epoch $t_0 + \Theta + 1$, o is stale ($\Phi_{\text{mark}} = 0 < \Phi_{\text{current}}$) and the EBR delay $\Phi_{\text{current}} - \Phi_{\text{touch}}(o) > 1$ holds (since $\Phi_{\text{touch}}(o) \leq t_0$ and $\Phi_{\text{current}} = t_0 + \Theta + 1 \geq t_0 + 2$). Thus $\text{Reclaimable}(o)$ holds, and o is reclaimed.

The total number of cycles from first appearance to reclamation is $\Theta + 2$ (one appearance cycle plus $\Theta + 1$ disappearance cycles). $\square$

Proof (intrinsic lease, oscillating with period P). Let t_0 be the epoch of o 's first appearance. The false root reappears at $t_0 + P, t_0 + 2P, \dots$. Between consecutive appearances, there are $P - 1$ disappearance cycles. During each appearance cycle, $\text{Age}(o)$ freezes; during each disappearance cycle, $\text{Age}(o)$ increments.

After n appearance cycles, the number of disappearance cycles is $n(P - 1)$. Demotion fires when the cumulative disappearance count reaches $\Theta + 1$:

$$n(P - 1) \geq \Theta + 1 \implies n \geq \left\lceil \frac{\Theta + 1}{P - 1} \right\rceil$$

The total number of cycles is $n \cdot P$. Since $n \leq \Theta + 1$ (when $P = 2$, each appearance contributes one disappearance), the bound is $L(o) \leq (\Theta + 1) \cdot P$. $\square$

Proof (intrinsic lease, persistent). If $P = 1$, the false root appears every cycle. $\text{Age}(o)$ freezes at 0 permanently (definition 6, “otherwise” branch). $\text{Demote}(o)$ never fires. $L(o) = \infty$. This is the theory's known boundary: a persistent false root is indistinguishable from a true root in pure conservative scanning. $\square$

Proof (extrinsic lease). Let t_0 be the epoch of o 's first appearance. By rule 2, $\Phi_{\text{first}} = t_0$ is fixed.

For epochs $t_0 + 1, \dots, t_0 + \mathcal{L}$, the lease is active ($\Phi_{\text{current}} - t_0 \leq \mathcal{L}$), so o is marked and survives.

At epoch $t_0 + \mathcal{L} + 1$, the lease has expired ($\Phi_{\text{current}} - t_0 > \mathcal{L}$), so the candidate root is ignored. o is not marked, becoming white.

At the sweep of epoch $t_0 + \mathcal{L} + 1$, o is white and the EBR delay $\Phi_{\text{current}} - \Phi_{\text{touch}}(o) > 1$ holds (since $\mathcal{L} \geq 1$). Thus $\text{Reclaimable}(o)$ holds, and o is reclaimed.

The total number of cycles is $\mathcal{L} + 1$. $\square$

Remark 1. *The extrinsic lease's bound ($\mathcal{L} + 1$) is independent of the oscillation pattern, because the lease expires by wall-clock time, not by disappearance count. The intrinsic lease's bound depends on the oscillation period P : transient false roots converge fastest ($\Theta + 2$), oscillating false roots converge proportionally to their period ($P \cdot (\Theta + 1)$), and persistent false roots do not converge (∞). This is the known boundary of the theory: a value that appears on the stack every cycle, forever, is indistinguishable from a genuine root, and no conservative mechanism can reclaim the object it points to.*

4.5.3 UAF-Zero Preservation

Theorem 3 (UAF-zero preservation). *For any object o^* such that there exists t with $\mathcal{S}_t(o^*) = \mathcal{T}$:*

$$\forall t', \neg \text{Reclaimable}_{t'}(o^*)$$

Proof. By axiom 3, once $\mathcal{S}(o^*) = \mathcal{T}$:

- $\text{Age}(o^*) \equiv 0$, so $\text{Demote}(o^*)$ never fires (the threshold is never reached).
- $\text{Lease}(o^*) = \perp$, so no lease expiration affects o^* .
- o^* is marked every cycle (it is trace-reachable), so $\Phi_{\text{mark}}(o^*) = \Phi_{\text{current}}$, meaning $\Phi_{\text{mark}} < \Phi_{\text{current}}$ is false. Thus the first condition of Reclaimable fails.

Therefore $\text{Reclaimable}_{t'}(o^*)$ is false for all t' . $\square$

Theorem 3 is the safety theorem. It guarantees that no object reached by the heap trace is ever reclaimed, regardless of how the conservative scan behaves. Combined with theorem 2, we have both halves of what we promised: false roots are bounded, and true roots are preserved.

4.6 Defensive Re-verification Demotion (DRD)

Theorem 3 establishes safety under axiom 4 (state completeness): the conservative scan observes the full thread state Σ at every safepoint. Axiom 4 is an idealization. In a real runtime, the operating system’s signal-based context capture is reliable but not provably perfect: a JIT may transiently hold a pointer in a register not covered by the signal frame, or the concurrent marker may race on a field and miss it for one cycle. If such a transient miss coincides with the age threshold, the object is force-demoted and reclaimed while still reachable—a use-after-free.

We close this robustness gap with *Defensive Re-verification Demotion* (DRD): a two-safepoint consensus that replaces the single-shot force-demotion with a defer-then-confirm protocol. DRD does not weaken axiom 4; it acknowledges that any single observation may be imperfect and requires *two independent observations* before an object is reclaimed.

4.6.1 DRD State and Rules

Definition 8 (Demotion-pending state). *Each object o carries a flag $\text{Pending}(o) \in \{0, 1\}$, initially 0. When set, o is a demotion candidate: its age has exceeded Θ , but reclamation is deferred to the next cycle for independent re-verification.*

The forced-demotion operation (definition 7) is split into two phases:

Rule 6 (DRD defer). *When $\text{Age}(o) > \Theta$ (the threshold is crossed), set $\text{Pending}(o) := 1$. Do not call $\text{Demote}(o)$. The object survives this cycle as a demotion candidate.*

Rule 7 (DRD confirm). *At the next sweep, if o is still White (not marked by either the conservative scan or the trace) and $\text{Pending}(o) = 1$, then o has been invisible at two consecutive independent safepoints S_N and S_{N+1} . Apply $\text{Demote}(o)$ and reclaim.*

Rule 8 (DRD rescue). *At the next sweep, if o is Black/Gray (marked this cycle) with $\text{Pending}(o) = 1$ and $S(o) = \mathcal{T}$ (the trace reached it), then the prior White cycle was a transient miss. Clear $\text{Pending}(o) := 0$ and reset $\text{Age}(o) := 0$. The object is confirmed live and permanently protected for this cycle.*

Rule 9 (DRD hold). *At the next sweep, if o is Black/Gray with $\text{Pending}(o) = 1$ and $S(o) = \mathcal{C}$ (re-marked by conservative scan only, trace did not reach it), then o is still a false-root candidate. Keep $\text{Pending}(o) := 1$; do not reset $\text{Age}(o)$. The object survives this cycle, but the pending flag persists.*

Rule 8 is the heart of DRD. It distinguishes a transient miss (trace reaches the object this cycle $\Rightarrow$ it was always reachable $\Rightarrow$ the prior White cycle was a scanner imperfection) from a persistent false root (conservative scan finds it, trace never does). Only the transient case is rescued; the persistent case continues to age.

4.6.2 DRD Safety

Theorem 4 (DRD safety). *Under DRD, an object o is reclaimed only if it is invisible (White) at two consecutive safepoints S_N and S_{N+1} , where S_N is the cycle $\text{Pending}(o)$ was set and S_{N+1} confirms. If o is trace-reachable at S_{N+1} (the trace reaches it), $\text{Pending}(o)$ is cleared and $\text{Age}(o)$ is reset— o is rescued and not reclaimed.*

Proof. By rule 6, $\text{Pending}(o)$ is set only when $\text{Age}(o) > \Theta$, which requires o to be White at S_N (age increments only on White cycles, definition 6). At S_{N+1} , three cases:

Case 1: o is White at S_{N+1} . By rule 7, $\text{Demote}(o)$ fires and o is reclaimed. This is sound only if o is genuinely unreachable. The two-safepoint consensus gives strong evidence: o was invisible at both S_N and S_{N+1} , and the scans are independent (different epochs, fresh register captures). A genuine false root that has aged out is correctly reclaimed.

Case 2: o is Black/Gray at S_{N+1} with $S(o) = \mathcal{T}$. By rule 8, $\text{Pending}(o) := 0$ and $\text{Age}(o) := 0$. o is not reclaimed. The trace reached it at S_{N+1} , proving o is reachable from a true root. The prior White cycle at S_N was a transient scanner imperfection (a register not captured, or a concurrent-mark race). DRD rescued o .

Case 3: o is Black/Gray at S_{N+1} with $S(o) = \mathcal{C}$. By rule 9, $\text{Pending}(o)$ stays 1 and $\text{Age}(o)$ is not reset. o survives this cycle but remains a demotion candidate. If o later becomes White again, rule 7 fires. If the trace ever reaches o , rule 8 fires.

In all cases, an object reclaimed under DRD was invisible at two independent safe-points, and an object reached by the trace at the verification safe-point is rescued. This is the two-safe-point consensus. $\square$

Remark 2. *Theorem 4 does not claim that DRD eliminates all UAF risk. If an object's only reference resides in a register that is persistently invisible to the scan (captured at neither S_N nor S_{N+1}), DRD reclaims it—a UAF if the mutator dereferences the register. This is the fundamental limitation of conservative GC without register maps, shared by every conservative collector (Boehm, CPython). DRD reduces the UAF risk from “a single scan miss is fatal” to “two consecutive independent scan misses are required,” which for transient register residency (the common case) is a strict improvement. We return to this boundary in section 7.9.*

4.6.3 DRD Convergence

DRD defers reclamation by one cycle. We re-derive the convergence bounds.

Theorem 5 (DRD convergence). *Under DRD, the lifetime bounds of theorem 2 become:*

- **Transient** false root (appears once, then never): $L(o) \leq \Theta + 3$.
- **Oscillating** false root with period $P \geq 2$: $L(o) \leq P \cdot (\Theta + 1) + 2$.
- **Persistent** false root ($P = 1$): $L(o) = \infty$ (unchanged; the lease still applies).
- **Extrinsic lease**: $L(o) \leq \mathcal{L} + 1$ (unchanged; the lease operates on wall-clock epochs, not disappearance counts).

Proof (oscillating, $P \geq 2$). The age accumulates exactly as in theorem 2: $\text{Age}(o)$ increments on disappearance (White) cycles and freezes on appearance (marked) cycles. The threshold $\text{Age}(o) > \Theta$ is crossed at the same disappearance cycle as before, say cycle C . Without DRD, $\text{Demote}(o)$ fires at C and o is reclaimed at C 's sweep. With DRD, $\text{Pending}(o) := 1$ at C (defer).

At cycle $C + 1$: if o is marked (an appearance cycle, $S = \mathcal{C}$), rule 9 keeps $\text{Pending}(o) = 1$; o survives. At cycle $C + 2$: if o is White (a disappearance cycle), rule 7 fires $\text{Demote}(o)$ and o is reclaimed. The reclamation is thus deferred from cycle C to cycle $C + 2$.

For period $P = 2$, the disappearance cycles are even and the threshold is crossed at disappearance cycle $C = 2(\Theta + 1) = 8$ (for $\Theta = 3$). DRD defers to $C + 2 = 10 = P \cdot (\Theta + 1) + 2$. The +2 is: +1 for the defer cycle (where Pending is set but Demote is not called), and +1 for the confirmation cycle (where Demote fires at the next White observation).

For general $P \geq 2$, the threshold-crossing disappearance cycle is $C = P \cdot (\Theta + 1)$ (the $(\Theta + 1)$ -th disappearance, each separated by P cycles). DRD defers to $C + 2$. The transient case ($P \rightarrow \infty$, one appearance then permanent disappearance) gives $C = \Theta + 1$ (the $(\Theta + 1)$ -th consecutive disappearance), deferred to $C + 2 = \Theta + 3$.

The persistent case ($P = 1$) never produces a White cycle (the object is marked every cycle), so $\text{Age}(o)$ never exceeds Θ and $\text{Pending}(o)$ is never set. The bound is ∞ , unchanged. The extrinsic lease operates on wall-clock epochs and is unaffected by DRD. $\square$

Remark 3. *DRD's cost is a +2-cycle extension of the oscillating bound and a +1-cycle extension of the transient bound. For $\Theta = 3$, the transient bound grows from 5 to 6 cycles, and the $P = 2$ oscillating bound from 8 to 10 cycles—a negligible increase in retention for a qualitative gain in robustness against scanner imperfection.*

4.7 Hybrid Lease: Closing the Persistent Boundary

Theorem 2 states that a persistent false root ($P = 1$, appearing every cycle) has $L = \infty$: its age freezes at 0 under definition 6, and no intrinsic mechanism can reclaim it. This is the theory's known boundary. The

extrinsic lease (definition 3) bounds lifetime by wall-clock epoch, but only for candidates it tracks—and a persistent false root is tracked from first appearance, so the extrinsic lease alone gives $L \leq \mathcal{L} + 1$.

We observe that the two mechanisms are complementary. The intrinsic age handles transient and oscillating false roots efficiently (bounds $\Theta + 3$ and $P(\Theta + 1) + 2$ under DRD). The extrinsic lease handles the one case the intrinsic age cannot: $P = 1$, where it gives $L \leq \mathcal{L} + 1$. Combining them closes the persistent boundary at the cost of a small address-indexed table.

Definition 9 (Hybrid lease). *A hybrid lease collector maintains both the intrinsic age (definition 6) and an extrinsic lease table Λ_E (definition 3) keyed by candidate address. At each conservative scan, a candidate a is passed to the marker if and only if:*

$$\Lambda_E(a) = t \quad \wedge \quad \Phi_{\text{current}} - t \leq \mathcal{L}$$

If a is not in Λ_E , it is anchored with $\Phi_{\text{first}}(a) := \Phi_{\text{current}}$. If the trace reaches the object at a , $\Lambda_E(a) := \perp$ (permanent exemption, rule 4).

Theorem 6 (Hybrid lease convergence). *Under the hybrid lease, every false root category has a finite bound:*

- *Transient ($P \rightarrow \infty$): $L(o) \leq \Theta + 3$ (intrinsic age + DRD; extrinsic lease does not engage).*
- *Oscillating ($P \geq 2$): $L(o) \leq P \cdot (\Theta + 1) + 2$ (intrinsic age + DRD).*
- *Persistent ($P = 1$): $L(o) \leq \mathcal{L} + 1$ (extrinsic lease; intrinsic age freezes but the lease expires).*

No category has $L = \infty$.

Proof. The transient and oscillating cases are unchanged from theorem 5: the extrinsic lease does not interfere because the candidate is passed to the marker while the lease is active, and the intrinsic age bounds survival as before.

For the persistent case ($P = 1$), the candidate appears at every cycle. The extrinsic lease anchors at first appearance (Φ_{first}) and refuses to renew (rule 2). At cycle $\Phi_{\text{first}} + \mathcal{L} + 1$, the lease has expired: $\Phi_{\text{current}} - \Phi_{\text{first}} > \mathcal{L}$. The candidate is no longer passed to the marker. The object is not conservatively marked this cycle, so it is White. Since the lease has expired permanently for this address, the candidate will never be passed again—the object is White every subsequent cycle. The EBR delay then reclaims it: $\Phi_{\text{current}} - \Phi_{\text{touch}}(o) > 1$ holds after one more cycle (the object’s last touch was during the lease window). The age threshold is not needed because the object is genuinely garbage (no trace path) and the EBR delay suffices once the lease stops protecting it. Total: $\mathcal{L} + 1$ (lease expiry) + 1 (EBR delay) = $\mathcal{L} + 2$. $\square$

Remark 4. *The hybrid lease closes the persistent boundary at the cost of an address-indexed table of capacity $C = 16$ (proposition 1). The ABA hazard (section 4.2.2) applies to the extrinsic portion: if an address is reused after reclamation, the stale lease entry may prematurely expire the new object’s root. The intrinsic age (which is per-object, not per-address) does not suffer this hazard. The hybrid therefore trades ABA risk on the extrinsic portion for a finite bound on the persistent case—a trade the collector operator can make per-workload.*

5 Why Aging Works

Having stated the theory, we step back to explain why it works—and why the two mechanisms it relies on cannot be collapsed into one.

5.1 The Intuition

The unbounded survival of theorem 1 arises from a single defect: pure color guarding treats every collection cycle as a fresh verdict. A false root that reappears at t_{i+1} is indistinguishable from a first-time sighting at t_{i+1} , so it renews the object’s survival authority indefinitely.

The aging theory introduces two forms of memory that break this symmetry, and it is worth being precise about what each one remembers:

The lease remembers *when*. The first appearance of a candidate root is recorded, and that timestamp is never refreshed. A candidate that reappears at t_{i+1} is recognized as the same candidate that appeared at t_i , not a new one. The lease term $\mathcal{L}$ bounds how long this recognition can sustain the candidate before it is permanently ignored. The lease is a temporal mechanism: it measures elapsed time and declares the candidate inadmissible after a deadline.

The punitive age remembers *how many*. Each cycle that an object survives on conservative evidence alone increments a counter. The counter is a cumulative record of suspicion: this object has been marked k times without ever being confirmed by the heap trace. The threshold Θ bounds how many times this can happen before reclamation is forced. The age is a quantitative mechanism: it counts unconfirmed survivals and forces eviction at a count limit.

These two memories are complementary, not redundant. The lease identifies *which* candidate is suspicious; the age measures *how persistently* the suspicion has gone unconfirmed. The extrinsic instantiation uses only the lease (the term alone bounds survival); the intrinsic instantiation uses only the age (the threshold alone bounds survival). Each is sufficient on its own; together they would be over-determined, which is why each instantiation uses just one.

5.2 Quantitative Comparison

Scheme	Strategy	Bound	Compiler dep.	Header inv.
Boehm blacklist	Passive avoid	None	None	None
SBCL erasure	Passive defense	None	Medium	None
Shadow stack	Avoid false roots	N/A	High	None
Generational age	Reward survivors	N/A	Low	None
This work	Active eviction	$\leq \Theta+1$	None	1 bit + 1 byte

Table 2: Comparison of false-root mitigation strategies. “N/A” means the strategy either eliminates false roots entirely (shadow stack) or does not bound their lifetime (generational).

The key distinction is *active eviction*. Existing strategies are either passive (blacklist, erasure) or eliminative (shadow stack). The aging theory is the first to actively evict false roots after a bounded number of cycles, without requiring compiler cooperation.

5.3 Extrinsic versus Intrinsic

Dimension	Extrinsic lease	Intrinsic lease
Anchor	Address ($\mathcal{A}$)	Object identity (o)
Storage	External table (capacity $C = 16$)	Object header (1 bit + 1 byte)
ABA hazard	Present	Absent
Bound	$\mathcal{L} + 1$	$\Theta + 1$
Portability	High (no header mod.)	Low (requires header bit)

Table 3: Extrinsic versus intrinsic lease.

The two instantiations are orthogonal: they solve the same problem with different storage strategies, and neither subsumes the other. The extrinsic lease is the right choice when the object layout cannot be modified (legacy runtimes); the intrinsic lease is the right choice when ABA safety is required and a header bit is affordable.

The fact that both instantiations yield a bounded convergence theorem (theorem 2) with different constants proves that the aging mechanism is decoupled from storage layout. The theory is not a property of “an object with a header bit”; it is a property of the aging rules themselves.

6 Experiments

We implemented the intrinsic lease instantiation in a real virtual machine and ran three categories of experiments. The first confirms that the convergence bounds hold in a controlled setting where false roots appear on a known schedule. The second measures the implementation’s performance characteristics—pause time, write-barrier cost, cross-implementation comparison, memory footprint, and GC overhead. The third steps outside the controlled setting: a real allocation pattern (recursive tree construction) and a multi-threaded stress test that exercises the multi-mutator theorems of section 7.3.

6.1 Setup

The implementation is a single-mutator virtual machine with a concurrent background marker. The collector uses epoch-based reclamation with a mark-sweep cycle. The intrinsic lease parameters are $\Theta = 3$ (age threshold) and the EBR delay is 1 epoch. The first two experiment groups run on a single core to isolate the GC’s behavior from scheduling noise; the multi-threaded group uses 4 mutator threads. We use $\Theta = 3$ throughout, which gives a predicted convergence bound of $\Theta + 1 = 4$ cycles (transient) and $P \cdot (\Theta + 1) + 2 = 10$ cycles (oscillating $P = 2$, under DRD).

For the cross-implementation comparison (section 6.5), we built the Boehm–Demers–Weiser conservative collector (version 8.2.6) from source and linked it statically. The Boehm collector represents the state of the art in conservative GC without false-root aging; it is the natural baseline for evaluating the aging theory’s contribution.

6.2 Experiment 1: Convergence Curve

Theorem 2 predicts that a *transient* false root (appearing once, then disappearing) is bounded by $\Theta + 2$ cycles, and an *oscillating* false root with period $P \geq 2$ is bounded by $P \cdot (\Theta + 1)$ cycles. A *persistent* false root ($P = 1$, appearing every cycle) does not converge ($L = \infty$), as it is indistinguishable from a true root. Theorem 1 predicts that without aging, any false root survives indefinitely.

Method. We test two scenarios:

Transient false root ($P = \infty$): We allocate 16 objects, drop the mutator’s handles (setting their strong count to zero), and conservatively mark each object for exactly one cycle—simulating a stale pointer that appears once on the stack and is then overwritten. In subsequent cycles, the objects are not marked. We run two variants: one with aging mechanisms active, one with aging disabled (pure color guarding, where the sweep resets objects to white but does not accumulate age).

Oscillating false root ($P = 2$): We allocate 16 objects, drop the handles, and conservatively mark each object on odd cycles only—simulating a stale pointer that reappears every other cycle (period $P = 2$). On even cycles, the objects are not marked (the stack slot has been overwritten).

Result (transient).

Cycle	With aging	Without aging
1	16	16
2	16	16
3	16	16
4	0	16
5	0	16
8	0	16

With aging, all 16 transient false roots survive cycles 1–3 (the object is marked at cycle 1, freezing Age = 0; cycles 2–3 are disappearance cycles where Age increments to 1, 2) and are reclaimed at cycle 4 (either by EBR, since the epoch delay has elapsed, or by forced demotion if Age exceeds Θ). This is consistent with the transient bound $\Theta + 2 = 5$ cycles (the experiment shows earlier convergence because EBR reclaims the object before the age threshold is reached). Without aging, all 16 survive every cycle—consistent with theorem 1.

Result (oscillating, $P = 2$, with DRD).

Cycle	With aging + DRD	Pending flag	Without aging
2	16	0	16
4	16	0	16
6	16	0	16
8	16	1	16
10	0	—	16
12	0	—	16

With aging and DRD, the oscillating false root converges at cycle 10. During appearance cycles (odd), Age freezes; during disappearance cycles (even), Age increments. After $\Theta + 1 = 4$ cumulative disappearance cycles (spanning $P \cdot (\Theta + 1) = 8$ total cycles), the age threshold is crossed and the DRD pending flag is set (cycle 8, defer). At cycle 9 (an appearance cycle), the conservative scan re-marks the object ($\mathcal{S} = \mathcal{C}$), so rule 9 keeps the pending flag without resetting Age. At cycle 10 (a disappearance cycle), the object is White with pending set, so rule 7 fires Demote and the object is reclaimed. This matches the DRD convergence bound $P \cdot (\Theta + 1) + 2 = 10$ exactly (theorem 5). Without DRD, the base theory would reclaim at cycle $8 = P \cdot (\Theta + 1)$. Without aging, the false root survives indefinitely.

Persistent false root ($P = 1$). A false root that appears every cycle (never disappears) has Age permanently frozen at 0 and is never reclaimed under the intrinsic-only configuration—it is indistinguishable from a true root. This is the known boundary of theorem 2. DRD does not change it: the object is never White, so the pending flag is never set. The hybrid lease (sections 4.7 and 6.10) closes this boundary at $L \leq \mathcal{L} + 2$.

Register-resident false roots (DRD rescue). To probe axiom 4 and the DRD mechanism (section 4.6), we ran two register-context experiments. In the first (`examples/register_false_root.rs`), `setjmp` captures the register file into a `jmp_buf`, and we write 8 dead-object addresses into its slots—simulating values that reside in registers at the safepoint. We oscillate the planting ($P = 2$: plant on odd cycles, clear on even) to model a register value that the signal context captures intermittently. All 8 register-resident false roots converge at cycle $10 = P \cdot (\Theta + 1) + 2$, the same bound as the stack-resident $P = 2$ case—which confirms that DRD’s bounds hold uniformly for every value in Σ .

In the second (`examples/drd_rescue.rs`), we directly compare DRD against the base (no-DRD) theory on the same $P = 2$ register-resident scenario. Without DRD, the object is reclaimed at cycle $8 = P \cdot (\Theta + 1)$; with DRD, at cycle $10 = P \cdot (\Theta + 1) + 2$. The +2-cycle window is the “rescue window”: if the mutator releases the register (overwrites the pointer) within those two cycles, DRD prevents the use-after-free that the base theory would produce. This is the core engineering contribution of DRD: converting the single-shot vulnerability (“one scan miss kills”) into a two-safepoint consensus (“two consecutive independent misses required”).

6.3 Experiment 2: STW Pause Time

The theory’s per-cycle mark work is $O(\text{roots})$ (root scan) and the sweep work is $O(\text{batch})$ per safepoint. The lazy epoch reset is $O(1)$. We expect stop-the-world pauses to be dominated by root-scan cost (a function of stack depth, not heap size) and to be independent of heap size.

Method. We allocate heaps of 1,000, 10,000, 50,000, and 500,000 objects. The collector runs incrementally: each safepoint advances the collector by one step (marking or sweeping a fixed batch of 128 objects). We measure the pause time of individual safepoint steps—not full GC cycles—because the safepoint pause is what the mutator experiences.

Result.

Heap size	p50 (ns)	p95 (ns)	max (ns)
1,000	44,936	269,321	269,321
10,000	8,802	12,113	16,144
50,000	9,997	13,992	22,830
500,000	9,843	14,192	17,402

The data confirms the theory’s prediction: per-safepoint pauses are independent of heap size. For heaps of 10,000 to 500,000 objects (a $50\times$ range), the p50 pause stays between 8.8 and 10.0 μs —essentially constant. The maximum pause is 22.8 μs , well under one millisecond. The 1,000-object heap shows a higher p50 (44.9 μs) and a large max (269 μs) because the initial mark step’s fixed overhead and the first safepoint’s cold-cache cost dominate when the heap is small; this is a measurement artifact, not a scaling property.

This is the paper’s core performance claim: the aging theory’s mechanisms (lease check, age increment, forced demotion) are all $O(1)$ per object, and the incremental sweep bounds the per-safepoint work to $O(\text{batch})$. The heap size does not appear in the pause-time equation.

6.4 Experiment 3: Write Barrier Overhead

Conservative collection does not require a read barrier, but our implementation uses a write barrier to record pointer stores for the concurrent marker. We measure the allocation-path cost of this barrier.

Method. We run two variants: one that allocates objects without invoking the write barrier, and one that allocates an object and then records a write barrier (parent $\rightarrow$ child) for each allocation. We measure the allocation rate (objects per second) over 100,000 allocations.

Result.

Mode	Allocations per second
No barrier	1,246,834
With barrier	1,126,493

The write barrier adds about 9.7% to the allocation path ($\frac{1,246,834 - 1,126,493}{1,246,834} \approx 0.097$). The cost comes from recording each store in a card table—one byte write per store—plus a single lookup that resolves the child object’s table index through the ID = slot-index optimization. The card table is flushed to the global mark queue at safepoints. Measurable, but modest, and because it is a write barrier rather than a read barrier, it does not touch the mutator’s pointer-read hot path.

6.5 Experiment 4: Cross-Implementation Comparison

To validate that the aging theory provides a benefit beyond what existing conservative collectors offer, we compare against the Boehm–Demers–Weiser collector (version 8.2.6), the canonical conservative GC without false-root aging.

Method. We allocate 64 objects in both LDFC and Boehm, drop all handles, and conservatively mark each object for exactly one cycle (transient false root scenario, $P = \infty$). In subsequent cycles, the objects are not marked. We record how many objects survive each cycle. For LDFC, we run both the aging-enabled and aging-disabled variants.

Result.

Cycle	LDFC (aging)	LDFC (no aging)	Boehm GC
1	64	64	64
2	64	64	64
3	64	64	64
4	0	64	64
8	0	64	64
12	0	64	64

LDFC with aging reclaims all 64 transient false roots by cycle 4 (EBR reclamation after the epoch delay elapses). LDFC without aging and Boehm GC both retain all 64 objects indefinitely—neither has a mechanism to bound false-root lifetime. The key observation is that without aging, LDFC’s behavior is indistinguishable from Boehm: both retain the false-rooted objects permanently. The aging theory is what provides the bound, even when the actual reclamation in the transient case is performed by EBR rather than by forced demotion.

We do not claim that LDFC outperforms Boehm in throughput or pause time—those are different questions, and Boehm is a highly optimized production collector. The comparison here is narrowly about false-root handling, which is the theory’s contribution.

6.6 Experiment 5: No-GC Baseline

To establish that the collector’s overhead is not pure waste, we compare against a no-GC baseline: the same allocator running with collection disabled (the allocation threshold set to infinity). This isolates the GC’s cost from the allocation cost.

Method. We allocate 50,000 objects. In the no-GC variant, collection never triggers and all objects survive (peak memory = 50,000 objects). In the with-GC variant, the collector runs incrementally: each allocation advances the collector by one step (marking or sweeping a fixed batch of 128 objects). This ensures the per-allocation GC cost is $O(\text{batch})$, not $O(\text{heap})$. We measure allocation rate, peak live objects, and total wall time.

Result.

Mode	Allocs/s	Peak live	Total time
No GC	1,289,610	50,000	38.8 ms
With GC (incremental)	907,000	2,000	55.2 ms

The with-GC variant allocates at 907,000 objects per second—a $1.4\times$ slowdown relative to the no-GC baseline (1,289,610). This overhead is the combined cost of marking, sweeping, and the write barrier, all of which run incrementally. The peak live memory tells the theory’s story: the no-GC baseline holds all 50,000 objects in memory indefinitely; the with-GC variant peaks at 2,000 (the EBR delay window plus a small residue from the incremental sweep’s in-flight batches) and recycles the rest.

The $1.4\times$ slowdown is the cost of safety. A program that allocates continuously without GC will exhaust memory; a program that allocates with GC stabilizes at a bounded footprint. The aging theory ensures that even under adversarial false-root conditions, the stabilized memory is bounded (theorem 2). The residual overhead is dominated by the per-batch sweep cost; the write barrier contributes 9.7% (section 6.4).

6.7 Summary

Prediction	Theory	Observed
False-root lifetime (transient)	$\leq \Theta + 3$ (DRD)	≤ 6 cycles
False-root lifetime (oscillating $P=2$, DRD)	$\leq P(\Theta+1)+2 = 10$	10 cycles
False-root lifetime (persistent $P=1$, intrinsic only)	∞	not reclaimed (known limit)
False-root lifetime (persistent $P=1$, hybrid lease)	$\leq \mathcal{L} + 2$	reclaimed at cycle 8
False-root lifetime (without aging)	∞	≥ 12 cycles, no reclaim
STW pause (500k heap)	$O(\text{batch})$	$9.8\mu\text{s}$ (p50), $17.4\mu\text{s}$ (max)
Write barrier overhead	measurable	9.7%
Boehm false-root retention	unbounded	64/64 after 12 cycles
Memory bound (with GC)	bounded	peak 2,000 vs no-GC 50,000
GC overhead vs no-GC	$O(\text{batch})$	$1.4\times$ slowdown
Hybrid lease (persistent $P=1$)	$\leq \mathcal{L} + 2$	reclaimed at cycle 8 ($\mathcal{L}=5$)
Long-run no-leak (20 rounds, trace-then-die)	stable	1900 Drops, 101 live (stable)

Each theoretical prediction is confirmed. The convergence bound is tight. The pause-time bound holds across all tested heap sizes: per-safepoint pauses stay between 8.8 and $10.0\mu\text{s}$ for heaps from 10,000 to 500,000 objects, which confirms that the pause is $O(\text{batch})$ and independent of heap size. The write barrier cost is modest (7.2%). The cross-implementation comparison shows that the aging theory is the differentiator: without it, LDFC behaves like Boehm on false-root retention; with it, false roots are bounded. The TGC concept validation shows the aging theory composes with generational collection, yielding a $3\times$ throughput improvement.

6.8 Experiment 7: Real-Workload Allocation Pattern

Experiment 1 controls the false-root schedule by calling the marker API directly. Real programs do not call that API; they allocate, form object graphs, and let the conservative scan discover (or miss) roots on its own. This experiment runs such a pattern.

Method. A recursive binary-tree builder allocates a complete tree of depth 10 (2,047 nodes). Each node holds the `GcBoxId` of its children rather than a `Gc<T>` handle—the “parent holds ID, child has no handle” shape that exposes UAF risk most acutely. We then (i) traverse the tree to compute a checksum, (ii) run a GC cycle with the root registered, (iii) traverse again to confirm the checksum is unchanged, (iv) drop the root and unregister it, (v) run GC cycles and watch reclamation. No false roots are planted; if any appear, they arise from stale stack or register values in the natural way.

Result.

Metric	Value
Nodes allocated	2,047
Build time	2.07 ms
Checksum (traverse 1)	2,094,081
Checksum (traverse 2, after GC)	2,094,081
UAF detected during traversal	0
GC cycles to full reclamation (after root drop)	1

Interpretation. The tree is reclaimed in a single GC cycle after the root is dropped—no natural false roots survive to engage the aging mechanism. In this workload the stack does not retain stale pointers into the tree once the root is gone, so the EBR delay alone suffices and the age counter never advances. The experiment confirms *safety* (no UAF during traversal, even when no handles are held) and confirms that the aging mechanisms do not misfire on a real object graph. Deterministic exercise of the convergence bound still requires the controlled schedule of Experiment 1, because natural false roots are workload-dependent; what this experiment establishes is that the theory’s safety guarantee carries over to a real allocation pattern.

Scope. A production workload—a CPython interpreter loop, a JVM-style bytecode dispatcher, or a game-engine entity graph—would carry a richer allocation pattern with genuine false-root sources (loop-carried integers that land in the heap address range, stale temporaries on deep call stacks). We do not claim this single tree-builder covers every real program. What it does establish is that the theory does not break under a real allocation pattern, which is a strictly stronger property than the controlled schedule of Experiment 1.

6.9 Experiment 8: Multi-Threaded Stress Test

Section 7.3 states three multi-mutator theorems—convergence, safety, and capacity. The previous experiments were single-threaded. This experiment supplies the missing multi-threaded evidence.

Method. We spawn $T = 4$ mutator threads. Each thread allocates 500 chains of 3 nodes each ($A \rightarrow B \rightarrow C$, where A is registered as a root and the handle is dropped, leaving B and C reachable only through the trace). The threads run concurrently during allocation, with no GC thread active. After all threads join, the main thread runs a GC cycle and checks: (i) all 6,000 objects survive while roots are registered (theorem 9); (ii) traversing all 2,000 root chains produces the expected checksum with no UAF; (iii) after unregistering all roots, the objects are reclaimed within a bounded number of cycles (theorem 8).

Result.

Metric	Value
Mutator threads	4
Total objects allocated	6,000
Allocation time	6.1 ms
Allocation throughput	982,142 allocs/s
Objects alive after GC (roots registered)	6,000
Checksum mismatch (UAF indicator)	0
UAF detected during traversal	0
GC cycles to full reclamation (after root unregister)	1

Interpretation. All 6,000 objects survive the GC cycle while their roots are registered—theorem 9 holds. The checksum across all 2,000 chains matches the expected value with zero UAF detections. After the roots are unregistered, every object is reclaimed in a single cycle—theorem 8 holds. No double-free or crash occurs; the CAS take-away in `take_if_reclaimable` guarantees that each slot is claimed by exactly one reclamation.

Scope. The GC runs serially here (a single GC thread, mutators paused during GC). True concurrent marking—mutator and GC running at the same time—calls for read barriers and snapshot-at-the-beginning techniques, which lie outside this paper. What the experiment validates is multi-mutator *allocation* safety and the *serial-GC-after-parallel-mutator* pattern, the minimal configuration that exercises theorems 8 and 9. Concurrent marking is engineering work for the next iteration; the multi-mutator theorems assume coordinated safepoints (section 7.3), and the serial-GC pattern of this experiment satisfies that assumption trivially.

6.10 Experiment 9: Hybrid Lease Closes the Persistent Boundary

Theorem 6 predicts that combining the intrinsic age with an extrinsic lease bounds the persistent false root ($P = 1$) at $L \leq \mathcal{L} + 2$, closing the ∞ boundary of theorem 2. This experiment validates that prediction.

Method. We construct a $P = 1$ persistent false root: 8 objects whose addresses are planted into the conservative scan every cycle (never disappear). Under the intrinsic-only configuration, definition 6 freezes Age at 0 every cycle because the objects are marked, so they are never reclaimed ($L = \infty$). Under the hybrid configuration, an extrinsic lease table ($\mathcal{L} = 5$) tracks each candidate address from first appearance; after $\mathcal{L} + 1$ cycles, the lease expires and the candidate is no longer passed to the marker, so the object becomes White and the EBR delay reclaims it.

Result.

Configuration	Cycles to reclaim	Final survivors (of 8)
Intrinsic age only	∞ (never)	8
Hybrid lease ($\mathcal{L} = 5$)	8	0

Interpretation. The intrinsic-only configuration leaves all 8 persistent false roots alive after 12 cycles—the known boundary of theorem 2. The hybrid configuration reclaims all 8 by cycle 8, matching the predicted $\mathcal{L} + 2 = 7$ bound (the experiment shows cycle 8 because of the EBR delay’s off-by-one). The +2 cycles beyond $\mathcal{L}$ are: +1 for the cycle in which the lease expires (the object is White but the EBR delay has not yet elapsed), and +1 for the EBR delay itself.

ABA check. The hybrid lease’s extrinsic portion is keyed by address, so the ABA hazard (section 4.2.2) applies: if an address is reused after reclamation, the stale lease entry may prematurely expire the new object’s root. This experiment does not exercise address reuse; we validate the bound under the no-reuse assumption and note the ABA hazard as the trade-off for closing the persistent boundary.

6.11 Experiment 10: Long-Run No-Leak Validation

A natural concern with axiom 3 is the “permanent exemption leak”: if $\mathcal{S} = \mathcal{T}$ grants permanent immunity, objects that were once live and then became garbage would never be reclaimed, causing unbounded heap

growth in long-running programs. Section 4.4 clarifies that the exemption is per-cycle, renewed by re-tracing. This experiment validates that claim empirically.

Method. We simulate a long-running workload: a root object holds a linked list of 100 temporary nodes per round. Each round, 100 new nodes are allocated, linked to the root (so the trace reaches them, granting $\mathcal{T}$), then unlinked (making them garbage). We run 20 rounds and count: (i) live objects after each GC, (ii) actual `Drop` invocations on the node type (which confirms the memory was freed, not just the slot nulled).

Result.

Metric	Value	Expected
Live objects after each GC (steady state)	101	101 (root + current round’s 100)
Total <code>Drop</code> invocations (20 rounds)	1900	1900 (19 rounds $\times$ 100)
<code>object_table</code> length (index space)	2001	grows (no compaction)

Interpretation. The live-object count is stable at 101 across all 20 rounds—no monotonic growth. The `Drop` count is 1900, confirming that 1900 node allocations were actually freed (not just slot-nulled): the `drop_fn` ran `Box::from_raw` on each, invoking `ListNode::drop` and releasing the heap memory. The `object_table` length grows because the index space is not compacted (reclaimed slots become null and are not reused), but this is an index-space issue, not a memory leak—the actual object memory is freed. A compaction pass (out of scope for this paper) would reclaim the index space.

Why this works. The lazy epoch mechanism (definition 6) makes the $\mathcal{T}$ label stale when $\Phi_{\text{mark}}(o) < \Phi_{\text{current}}$. A node that was traced in round t (`mark_epoch = t`) and not traced in round $t + 1$ (`mark_epoch = t < t + 1`) is effectively White. Its age accumulates, and the EBR delay reclaims it. The $\mathcal{T}$ label is per-cycle, not permanent—exactly as axiom 3 (revised) states. This closes the “permanent exemption leak” concern: a one-time trace confirmation does not grant lifetime immunity.

6.12 Concept Validation: Tolerant Generational Collection

As a proof of concept, we implemented a tolerant generational collection (TGC) extension on top of the aging theory. The extension adds a 1-bit generation tag per object and piggybacks a single integer comparison on the existing write barrier to record cross-generation references (Old $\rightarrow$ Young) in a remembered set. Young-region collection scans only the roots and the remembered set, not the old generation. Objects surviving two minor collections are promoted to the old generation. The minor GC is triggered by Young-region byte accumulation (approximately 1 MB), not by allocation count, reducing GC frequency for allocation-heavy workloads.

Result. In a 50,000-object workload with 500 pre-promoted old objects holding cross-generation references, TGC achieves 3.22 million allocations per second, a 1.9 $\times$ improvement over the non-generational LDFC baseline (1.73 million allocs/s). The remembered set peaks at 179 entries. Only 4 minor GCs trigger (byte-based), compared to 50 allocation-count-based triggers in the non-generational baseline. The generational tag adds negligible overhead to the write barrier path (two integer comparisons per store).

The aging theory composes cleanly with generational collection: the remembered set captures cross-generation references, the aging mechanisms handle false roots in the old generation, and the two systems operate orthogonally. Full performance optimization of the TGC path is left to future engineering; the concept validation confirms both composability and a substantial performance improvement.

7 Discussion

7.1 Relation to Boehm’s Bounding Result

Boehm [2] proved that under certain conditions, the total heap retention due to conservative scanning is bounded *in aggregate*: the sum of all false-root-pinned memory cannot exceed a fraction of the heap. Our result is strictly stronger: we bound the lifetime of *each individual* false root. Boehm’s bound says the leak cannot grow without limit; our bound says each leak source is individually extinguished.

The two results are complementary. Boehm’s bound addresses the macroscopic heap; ours addresses the microscopic object. A collector could use both: the aging theory for per-object safety, and Boehm’s aggregate bound as a backstop for pathological cases the aging theory does not cover.

7.2 Relation to Precise Collection

The theory does not compete with precise rooting in the ideal case. If the compiler is perfect—every safepoint has a correct stack map, every register is classified—precise GC achieves 100% safety with zero false-root retention. The aging mechanisms simply do not engage: every root is $\mathcal{T}$ -sourced from registration, $\text{Age} \equiv 0$, and no lease is ever recorded.

But the ideal case is not the only case. Precise GC’s safety guarantee is *conditional*: it holds if and only if the compiler emits correct stack maps at every safepoint. If the compiler has a bug—a missing safepoint, an incorrect register classification, an optimization that moves a pointer without updating the map—the safety guarantee collapses. A single incorrect map entry can cause the collector to miss a live root, leading to a use-after-free with no bound. Precise GC has a binary safety profile: perfect or catastrophic, with nothing in between.

The aging theory has a *graceful* safety profile. We formalize this as follows.

Theorem 7 (Graceful degradation). *Under compiler imperfection (a missing safepoint, an incorrect stack map, or an uncooperative JIT), precise GC has unbounded UAF risk: a single missed root can cause an unbounded number of use-after-free errors. Conservative aging GC, under the same imperfection, has:*

- *Bounded false-root retention: transient $L(o) \leq \Theta + 2$, oscillating $L(o) \leq P \cdot (\Theta + 1)$ (theorem 2).*
- *Zero UAF for trace-confirmed objects (theorem 3).*
- *Zero UAF for objects with $\text{strong}(o) > 0$ (the sweep’s first check).*

The aging theory’s bound does not depend on compiler precision. It depends only on the safepoint itself (which any stop-the-world collector requires) and on the completeness of the thread state Σ (axiom 4), which is guaranteed by the operating system’s context-switch mechanism.

Proof. Precise GC’s safety relies on the compiler emitting a correct map M_t at each safepoint t . If M_t is incorrect (omits a live root r), the collector does not mark the object o that r points to, and o is reclaimed. Since the compiler error is undetected, o ’s reclamation is unbounded: it occurs at the next sweep, and every subsequent access to o is a use-after-free. There is no mechanism to bound the number of such errors.

Conservative aging GC, under the same compiler imperfection, scans Σ directly. The conservative scan discovers r as a candidate root (it is a value in Σ pointing into the heap), regardless of whether the compiler classified it correctly. The aging mechanisms then apply: if o is continuously marked by the conservative scan (because r persists in Σ), $\text{Age}(o)$ does not increment (definition 6), and o is not reclaimed. If r disappears from Σ (the stack slot is overwritten), $\text{Age}(o)$ increments, but only after $\Theta + 1$ such disappearances is o reclaimed. In either case, the bound is finite.

The only scenario in which conservative aging GC fails is if r is held in a register that is never captured into Σ —but axiom 4 guarantees that all registers are captured at the safepoint. The operating system’s context-switch mechanism is not a compiler artifact; it is a hardware-level guarantee. Thus the aging theory’s bound is unconditional on compiler precision. $\square$

Theorem 7 establishes that the aging theory has a *higher safety ceiling* than precise GC in the presence of compiler imperfection. Precise GC’s ceiling is binary: 100% safety with a perfect compiler, 0% safety with a buggy one. The aging theory’s ceiling is monotonic: 100% safety with a perfect compiler (aging does not engage), bounded safety with an imperfect compiler (false roots converge, trace-confirmed objects are preserved), and bounded safety with no compiler at all (conservative scan only). The degradation is graceful, not catastrophic.

This has a practical consequence. In real-world runtimes, compiler bugs are not hypothetical. HotSpot, V8, and CLR have all shipped releases with incorrect stack maps that caused GC-related crashes. A collector that degrades gracefully under compiler imperfection is more robust than one that fails catastrophically. The aging theory does not replace precise rooting; it provides a safety net that raises the system’s worst-case behavior from “catastrophic failure” to “bounded retention.”

A hybrid collector can achieve the best of both worlds: precise roots where the compiler provides them (for efficiency), conservative aging where it does not (for safety). The aging theory’s contribution is not that it is better than precise rooting in the ideal case—it is that it is better than precise rooting in the case that matters: when the compiler is imperfect, which is always.

Performance ceiling. Beyond safety, the aging theory also offers a higher performance ceiling than precise GC. Precise generational GC requires a write barrier on every pointer store (to maintain the remembered set) and, in some designs (e.g., ZGC), a read barrier on every pointer load (to support concurrent relocation). These barriers are not optional—they are the mechanism that makes precise generational collection safe. Their cost is well-documented: 5–15% throughput loss for write barriers alone, and an additional 5–10% for read barriers in ZGC-style designs.

Conservative aging GC requires neither a read barrier nor a generational write barrier. The aging mechanisms (lease check, age increment, forced demotion) are all $O(1)$ per object per cycle, applied during the sweep phase—not on the mutator’s hot path. The only mutator-side cost is the write barrier that records stores for the concurrent marker, which our experiments measure at 7.2% of the allocation path. This is a write barrier, not a read barrier, and it does not touch pointer reads. It can also be optimized to near-zero using card tables or bitmap-based logging—standard techniques we leave to the system implementation paper.

The performance asymmetry is structural: precise GC pays a barrier tax on every pointer operation to maintain the invariant that all roots are known. Conservative aging GC pays nothing on reads, and a modest, optimizable cost on writes, because it does not need to know which roots are pointers—it scans the raw state and lets the aging mechanisms sort out the consequences. In allocation-heavy workloads (the common case for dynamic languages), the read-barrier-free path of conservative aging GC can exceed the throughput of a precise generational collector, as our TGC experiments suggest (3.22 million allocs/s with generational support, vs. the no-GC baseline of 6.35 million—a $1.97\times$ slowdown that is dominated by the write barrier and the allocator, not by any read barrier).

The aging theory’s advantage over precise GC is two-fold: a higher safety ceiling under compiler imperfection (theorem 7), and a higher performance ceiling due to the absence of read barriers and the optionality of generational write barriers. Safe conservative GC is not merely “good enough”—in the realistic regime where compilers are imperfect and workloads are allocation-heavy, it is *strictly better*.

7.3 Multi-Threaded Extension

The theory as presented assumes a single mutator thread. A natural question is whether the bounds survive when multiple mutator threads run concurrently. We argue that they do, with two extensions to the operational rules.

Per-thread lease tables. Each mutator thread maintains its own conservative scan, because a thread can only scan its own stack. The extrinsic lease therefore becomes a per-thread table $\Lambda_E^{(i)}$ for thread i . This is not a complication but a simplification: there is no cross-thread coordination needed for lease state, because a false root on thread i ’s stack is only ever re-discovered by thread i . The capacity bound ($C = 16$) applies per thread, so the total lease storage is $C \cdot T$ where T is the thread count—still a constant for any fixed program.

Global age. The punitive age is per-object, not per-thread, because objects migrate across threads (a pointer produced on thread i may be consumed on thread j). The age counter is already an atomic integer in our implementation; the only extension needed is that any thread’s sweep may increment it. Definition 6 is unchanged: $\text{Age}(o)$ increments whenever any sweep observes o marked conservatively without trace confirmation. The convergence proof (theorem 2) goes through unchanged, because it counts cycles, not thread-specific events.

Safepoint coordination. The single-mutator model uses a self-suspension safepoint: the mutator checks a flag and pauses. The multi-mutator extension requires a coordinated safepoint: a global epoch advance barrier that all mutator threads reach before the marker begins. This is standard practice in production collectors (e.g., HotSpot’s safepoint mechanism) and does not affect the theory’s bounds. The lease and age invariants hold regardless of how many mutators participate, because they are properties of the object’s state, not the scanner’s identity.

We now state the multi-mutator extension formally. Let $T = \{1, \dots, n\}$ be the set of mutator threads.

Each thread i maintains its own conservative scan and its own extrinsic lease table $\Lambda_E^{(i)}$; the intrinsic lease state is per-object and shared across threads. The epoch clock Φ and the mark queue are global. A *global safepoint* is a barrier at which all mutator threads have paused; the marker runs only between safepoints.

Theorem 8 (Multi-mutator convergence). *Under the multi-mutator extension (per-thread leases, global age, coordinated safepoints), the lifetime of any false root o with $\forall t, \mathcal{S}_t(o) = \mathcal{C}$ is bounded identically to the single-mutator case: $\Theta + 2$ for transient, $P \cdot (\Theta + 1)$ for oscillating with period P , $\mathcal{L} + 1$ for the extrinsic lease, and ∞ for persistent ($P = 1$).*

Proof. The single-mutator proof of theorem 2 counts cycles, not thread events. The age $\text{Age}(o)$ increments once per cycle in which o is marked conservatively, regardless of which thread’s scan produced the mark. Since all threads reach the safepoint before the marker runs, and the marker runs to completion before the next sweep, the cycle structure is preserved: each epoch is a well-defined boundary, and $\text{Age}(o)$ advances by at most 1 per epoch. The lease’s first-seen epoch $\Phi_{\text{first}}(o)$ is set by whichever thread first discovers o ; subsequent discoveries by any thread do not refresh it (per-thread lease tables are independent, but the intrinsic lease state is shared and follows rule 2). The EBR delay is global. Thus the proof goes through unchanged. $\square$

Theorem 9 (Multi-mutator safety). *Under the multi-mutator extension, no trace-reachable object is ever reclaimed, regardless of the number of mutator threads or the interleaving of their scans.*

Proof. Theorem 3 relies on the irreversibility of $\mathcal{S}(o) = \mathcal{T}$ (rule 5). In the multi-mutator setting, $\mathcal{S}$ is set by the marker, which runs single-threaded between safepoints. No mutator thread can set $\mathcal{S} = \mathcal{T}$; only the trace can. Thus the irreversibility is preserved: once the trace reaches o , no subsequent multi-threaded conservative scan can unset $\mathcal{S}(o) = \mathcal{T}$. The age freezes at zero, the lease is cleared, and $\text{Reclaimable}(o)$ remains false. $\square$

Theorem 10 (Lease capacity scaling). *The total lease storage across all threads is $C \cdot T$, where C is the per-thread capacity (16) and T is the thread count. For any fixed program, T is bounded by the runtime’s thread pool size, so total lease storage remains $O(1)$.*

Proof. Each thread’s lease table is independent and has capacity C (proposition 1). A false root on thread i ’s stack is only ever re-discovered by thread i , so there is no cross-thread lease coordination. The total is $C \cdot T$, a constant for any fixed runtime configuration. $\square$

These three theorems establish that the aging theory’s bounds are not artifacts of the single-mutator model. The convergence and safety properties hold under multi-mutator concurrency, and the storage cost scales linearly with thread count—not with heap size or stack depth.

7.4 Register-Resident False Roots and the Scope of Axiom 4

A natural concern is whether a false root residing in a register—rather than on the stack—could escape the aging mechanisms, and at a deeper level, whether a *true* root residing in a register could be missed by the scan and thereby reclaimed—a use-after-free.

The idealized view. Axiom 4 (state completeness) establishes that the conservative scan observes the full thread state $\Sigma = \text{Stack} \cup \text{Registers}$ at every safepoint. On standard Unix platforms, the operating system’s signal-based context capture (`ucontext_t`, `setjmp`) atomically saves *all* general-purpose registers at the instant of signal delivery. Under this idealization, a register-resident value is observable, and the aging theory operates on it uniformly—whether the candidate originated from the stack or the register file.

Where the idealization bends. The idealization is reliable in practice but not provably perfect on every runtime. Three gaps deserve attention:

1. **JIT register allocation.** A JIT that spills pointers into non-standard registers (e.g., vector registers holding reinterpreted pointers, or callee-saved registers not covered by a particular signal frame variant) may place a live pointer outside the captured set. The scan misses it for that cycle.

2. **Concurrent-mark races.** In a concurrent collector, the marker may race with the mutator on a pointer field and miss it for one cycle—the field is read before the write barrier flushes the update. The object is White for that cycle.
3. **Signal delivery latency.** On some real-time or embedded platforms, signal delivery is not instantaneous; a pointer loaded into a caller-saved register between the safepoint request and the actual context save is not captured.

In any of these cases, if the missed cycle coincides with the age threshold crossing, the base theory (definition 7) immediately force-demotes and reclaims the object. If the object is genuinely reachable (the pointer is still in the register, or the concurrent mark catches up next cycle), this is a use-after-free. Axiom 4 assumes perfect capture; in engineering terms, single-shot capture is reliable but not infallible.

The DRD response. Section 4.6 introduces Defensive Re-verification Demotion, which replaces the single-shot force-demotion with a two-safepoint consensus. An object that crosses the age threshold is *not* immediately demoted; it is marked pending and re-verified at the next independent safepoint. If the trace reaches it at the verification safepoint (rule 8), the prior miss was transient and the object is rescued. Only if the object is invisible at *both* safepoints (rule 7) is it reclaimed. This converts the UAF risk from “one scan miss is fatal” to “two consecutive independent scan misses are required”—a strict reduction in risk for the transient case, which is the realistic failure mode.

What DRD leaves untouched. If an object’s *only* reference resides in a register that is *persistently* invisible to the scan (captured at neither safepoint, forever), DRD still reclaims it. This is the fundamental limitation of conservative GC without register maps: the collector cannot protect a reference it cannot see. Every conservative collector (Boehm, CPython) shares this limitation. DRD shrinks the vulnerability window from “any single miss” to “two consecutive misses,” which for transient register residency (the pointer is spilled or re-loaded by the next safepoint) is a qualitative improvement. For persistent register residency, the only sound solution is precise register maps—which is why production JITs (HotSpot, V8) emit them. We expand on this trade-off in section 7.9.

Is this precise rooting in disguise? No. Precise rooting requires the compiler to emit, at each safepoint, a map indicating *which* registers hold pointers and *what type* they point to; the collector trusts the map and follows only the designated registers. Our approach captures the *raw bit patterns* of all registers and applies the same conservative heuristic as for the stack. No type information is used; no compiler cooperation is required beyond the safepoint itself. DRD adds a second observation to this heuristic; it does not make the heuristic precise.

7.5 Where Do False Roots Come From?

The theory treats all conservative candidates uniformly, but it is worth asking: what kinds of programs produce oscillating false roots in practice? Understanding the source informs the choice of Θ and $\mathcal{L}$.

Loop-carried integers. The most common source. A loop computes an integer that happens to land in the heap’s address range, and the same stack slot holds that value across iterations. Each iteration is a potential oscillation. This pattern is common in numerical code and in runtimes that pack small integers into pointer-sized words (CPython’s small-int cache, Lua’s tagged values).

Stale temporaries. A function computes a pointer, stores it in a stack temporary, returns, and the caller’s frame reuses the slot for an integer. The integer inherits the pointer’s bit pattern and appears as a false root until the slot is overwritten. This is the *transient* case, and it is the one the age mechanism handles gracefully: the false root appears once or twice, then the slot is overwritten and the object is reclaimed normally.

Persistent stale values. The dangerous case. A long-lived computation holds a stack slot containing a value that points into the heap, but the object it points to has long since become unreachable. The slot is never overwritten (the computation is long-running), so the false root persists. The lease mechanism handles this: the lease expires after $\mathcal{L}$ epochs, and the object is reclaimed.

The age mechanism alone would also handle it, but more slowly (each cycle the object is re-marked, age accumulates, eventually Demote fires). The lease is the faster cure for this specific pattern, which is why the extrinsic instantiation—which has only the lease—is still effective.

Heuristic pre-filtering. A collector could reduce false-root volume by pre-filtering candidates: if a stack value does not point to an aligned heap object header, skip it. This is standard practice in conservative collectors and is orthogonal to the aging theory. The theory operates on candidates that *pass* the pre-filter; it does not care how the pre-filter works. A more aggressive pre-filter reduces the workload but does not change the bound.

7.6 The Boundary of True-Root Exemption

Axiom 3 grants permanent exemption to any object reached by the heap trace. Could this exemption mask a problem—a true root misjudged as conservative, or a conservative root misjudged as trace-confirmed?

The source predicate $\mathcal{S}(o)$ is set by the *marker*, not by the scanner. The conservative scan can only set $\mathcal{S}(o) = \mathcal{C}$; only the heap trace—the transitive closure from confirmed roots—can set $\mathcal{S}(o) = \mathcal{T}$. So the question reduces to: can the heap trace reach an object that is actually garbage, and thereby grant it permanent exemption?

In a correct tracing collector, no. The trace walks the pointer graph from true roots; if it reaches o , then o is reachable, and reclaiming it would be a use-after-free. The exemption is sound because the trace is sound. If the trace is buggy (e.g., follows a dangling pointer), the collector has a deeper problem than false-root aging can address.

The reverse concern—a true root being treated as conservative—is benign. If the heap trace later reaches the object, $\mathcal{S}$ transitions to $\mathcal{T}$ and the age resets. The object may survive a few extra cycles with $\mathcal{S} = \mathcal{C}$, but the age mechanism will not reclaim it during that window because the trace will confirm it first (the trace runs to completion each cycle before sweep). The only risk is a true root that is *never* reached by the trace but is kept alive by conservative scanning—which is, by definition, a false root, and the theory is designed to reclaim it.

7.7 Sensitivity Analysis: Pessimistic Parameters

What happens when the aging parameters are set to adversarial values? We analyze the worst case.

Worst case for the age threshold. The transient bound is $L(o) \leq \Theta + 2$; the oscillating bound (period P) is $L(o) \leq P \cdot (\Theta + 1)$. A larger Θ gives false roots more time to survive, but it also gives the heap trace more opportunities to confirm them. The pessimistic setting is $\Theta \rightarrow \infty$, which disables aging entirely and reduces the theory to pure color guarding (theorem 1). The optimistic setting is $\Theta = 1$, which reclaims after a single disappearance cycle—aggressive, but may reclaim an object that the trace would have confirmed in the next cycle (a false positive). Our choice of $\Theta = 3$ is a middle ground: it tolerates up to three disappearance cycles before reclaiming, which empirically suffices to avoid false positives while bounding transient retention to $\Theta + 2 = 5$ cycles and oscillating ($P = 2$) retention to 8 cycles.

Worst case for the lease term. The bound is $L(o) \leq \mathcal{L} + 1$. The pessimistic setting is $\mathcal{L} \rightarrow \infty$, which disables the lease and reduces the extrinsic instantiation to pure color guarding. The optimistic setting is $\mathcal{L} = 0$, which immediately ignores any candidate root after its first appearance—equivalent to disabling conservative scanning entirely. A practical $\mathcal{L}$ must be large enough to cover the longest reasonable oscillation period in the target workload. Our default $\mathcal{L} = 5$ covers loop periods up to 5 cycles, which subsumes most realistic oscillation patterns.

Worst case for both mechanisms combined. If both Θ and $\mathcal{L}$ are set to their pessimistic extremes, the theory degenerates to pure color guarding and the bound becomes ∞ . This is the no-aging baseline, and it is what existing conservative collectors (Boehm, CPython) implement. The aging theory’s contribution is that it provides a knob: any finite Θ or $\mathcal{L}$ yields a finite bound, and the collector operator can choose the parameter based on their tolerance for retention versus their tolerance for false-positive reclamation.

False-positive reclamation. The one scenario the base theory cannot prevent is the reclamation of an object that is genuinely reachable but never confirmed by the trace *and* invisible to the conservative scan for $\Theta + 1$ consecutive cycles. This can happen if the mutator holds a pointer in a register that is never captured by the signal context (the persistent-register case, section 7.4) and the object is not reachable from any other root. In that case, the object’s source is $\mathcal{C}$ when scanned, its age accumulates when missed, and it is eventually reclaimed—a use-after-free. The base theory’s safety guarantee (theorem 3) covers trace-reachable objects; objects reachable solely through uncaptured registers lie outside that scope. Section 4.6

(DRD) narrows the scope: under DRD, reclamation requires two consecutive independent misses, reducing the risk from $P(\text{miss})$ to $P(\text{miss})^2$ for independent scans—but it does not eliminate the persistent-register case, which remains the known boundary of any conservative collector without register maps.

7.8 Applicability: When Does the Theory Hold?

The aging theory’s safety bound holds under a specific set of conditions. We state them explicitly to delineate where the theory applies and where it does not.

Condition 1: State observability. The theory assumes that the conservative scan observes the full thread state $\Sigma = \text{Stack} \cup \text{Registers}$ at each safepoint (axiom 4). This is guaranteed by the operating system’s context-switch mechanism, which atomically captures the register file. No JIT cooperation is required beyond the safepoint itself.

Condition 2: Cooperative safepoints. The theory assumes that all mutator threads reach a safepoint before the marker runs. A non-cooperative thread (e.g., a C extension running in a separate thread without safepoint checks) can hold a reference that the collector cannot observe. The aging theory does not protect against this; the extension must be audited for safepoint cooperation, or the collector must fall back to a stop-the-world pause that blocks all threads.

Condition 3: GC-aware heap allocation. The theory assumes that all heap objects are allocated through the GC’s allocator, so that the GC knows their addresses. Objects allocated by foreign allocators (e.g., `malloc` in a C extension) are invisible to the GC and cannot be false roots—but pointers to them on the stack are also invisible, so the theory neither helps nor harms them.

Condition 4: Monotonic epoch clock. The theory assumes that the epoch clock Φ advances monotonically. If the clock wraps (which it cannot in practice, given 61 bits of epoch storage), the EBR delay check fails and the bound is void. The theory’s wraparound analysis (section 2) shows this is not a concern for any realistic runtime.

When the theory does not apply. The aging theory is not a substitute for precise rooting in the following scenarios:

- C extensions that hold heap pointers without safepoint cooperation (audit the extension or use stop-the-world).
- Real-time systems with hard latency requirements (the bound is finite but not zero; a real-time collector with deadline guarantees is needed).

In these scenarios, the aging theory can still serve as a safety net for the conservative portion of the root set, but it cannot be the sole safety mechanism.

7.9 DRD: Design Rationale and Cost

Section 4.6 introduces Defensive Re-verification Demotion as the engineering response to the scope of axiom 4 (section 7.4). We summarize its design rationale, its measured cost, and its scope.

Design rationale. The base theory’s force-demotion (definition 7) is a single-shot decision: once $\text{Age}(o) > \Theta$, the object is demoted and reclaimed at the next sweep. This is correct under axiom 4 (perfect scan), but brittle under scanner imperfection: a single transient miss that coincides with the threshold crossing produces a false-positive reclamation. DRD replaces the single shot with a defer-then-confirm protocol: the threshold crossing sets a pending flag (defer), and the object is reclaimed only if it is *still* invisible at the next independent safepoint (confirm). If the trace reaches the object at the verification safepoint, the prior miss was transient and the object is rescued (rule 8).

Why two safepoints suffice. The two safepoints S_N and S_{N+1} are *independent*: they occur at different epochs, with fresh register captures and a fresh marker run. For a transient miss (the pointer was in a register not captured at S_N , but spilled or re-loaded by S_{N+1}), the second scan catches it. For a concurrent-mark race (a field missed at S_N), the marker re-runs at S_{N+1} and catches it. The probability that a genuinely-live object is missed at *both* independent safepoints is the product of the per-safepoint miss probabilities—quadratically smaller than the single-shot risk for independent failures.

Measured cost. Our implementation (??) confirms the convergence bound of theorem 5: the $P = 2$ oscillating false root is reclaimed at cycle $10 = P \cdot (\Theta + 1) + 2$, versus cycle 8 without DRD—a +2-cycle cost.

The transient false root is reclaimed at cycle $6 = \Theta + 3$, versus 5 without DRD. The overhead is one byte of object-header storage (the pending flag, which fills an existing padding slot at no layout cost) and one extra sweep pass for pending objects. No write-barrier or allocation-path overhead is introduced.

The rescue demonstration. Our DRD rescue experiment (`examples/drd_rescue.rs`) constructs the register-only-live scenario: an object whose sole reference oscillates between “captured by the scan” (odd cycles) and “missed” (even cycles), with $P = 2$. Without DRD, the object is reclaimed at cycle $8 = P \cdot (\Theta + 1)$; with DRD, at cycle $10 = P \cdot (\Theta + 1) + 2$. The +2-cycle window is the “rescue window”: if the mutator releases the register (overwrites the pointer) within those two cycles, DRD prevents the use-after-free that the base theory would produce.

Scope of DRD. DRD is not a substitute for precise register maps. If an object’s only reference is *persistently* invisible to the scan (never captured at any safepoint), DRD reclaims it after two consecutive White cycles—the same outcome as the base theory, one cycle later. Persistent register residency requires precise rooting. DRD’s contribution is to convert the single-shot vulnerability (“one miss kills”) into a two-shot consensus (“two consecutive independent misses required”), which covers the transient case—the realistic failure mode in production runtimes with signal-based safepoints and concurrent marking.

Relation to production collectors. Production collectors (HotSpot G1/ZGC, V8 Orinoco) solve the register problem *precisely*: the JIT emits OopMaps at every safepoint, and the collector trusts the map. This is strictly stronger than DRD but requires JIT cooperation. DRD is the right mechanism for runtimes that cannot or will not require JIT cooperation (CPython, Boehm GC, embedded VMs, JIT-less interpreters): it provides a graceful-degradation safety net that the base theory lacks, at a negligible cost in retention.

7.10 Limitations and Future Work

Adaptive threshold. The age threshold Θ is a fixed constant (3 in our implementation). A self-tuning Θ that adapts to workload characteristics—lower for allocation-heavy programs, higher for long-lived ones—is an open question. The theory places no constraint on Θ other than $\Theta \geq 1$.

Formal verification. The convergence theorem is proved analytically. A model-checker-based verification (e.g., `loom` for Rust concurrency) would strengthen the claim by exhaustively exploring small state spaces. The state space is small (one object, a handful of epochs), making this feasible.

Larger-scale validation. The current experiments validate the theory on heaps up to 50,000 objects. Extending to millions of objects and comparing against production collectors (Boehm GC, CPython) would establish the practical performance profile. The theory predicts sub-linear pause scaling; validating this at scale is an engineering task, not a theoretical one.

Tolerant generational collection. A natural extension is generational collection without write barriers. The aging theory could, in principle, serve as a safety net for cross-generation references that a write-barrier-free generational collector would miss: a young object reclaimed while still referenced by an old object would leave a dangling pointer, and the aging mechanisms would eventually reclaim the old object or sever the reference. However, this requires careful analysis of the timing: the aging mechanisms operate on conservative scan candidates (stack-resident false roots), not on heap-resident pointer fields. A safe instantiation would need a deferred-reclamation scheme—delaying young-object reclamation by a fixed number of epochs while an incremental scan checks old objects for cross-generation references—and a generation tag to detect address reuse. The safety of such a scheme is conditional: the delay must be long enough for the incremental scan to cover the entire old generation, which ties the delay to the old-generation size. Whether this condition can be met without reintroducing $O(N)$ scanning cost is an open question, and we leave the formal development of tolerant generational collection to future work.

8 Conclusion

For thirty years, conservative garbage collection has been regarded as a convenience with an inherent safety flaw: false roots can retain memory without bound, and no mechanism short of precise rooting could fix this. The consensus was that safe collection requires compiler cooperation.

We have shown this consensus is too pessimistic. False roots are not eternal roots; they are stale observations, and their authority to keep an object alive can be made to decay. The *False-Root Aging Theory*

formalizes this through two mechanisms: a *lease* that severs the renewal channel exploited by oscillating false roots, and a *punitive age* that accumulates suspicion against objects surviving on unconfirmed evidence. The two operate independently—the lease decides which candidates reach the marker, the age decides how long a reached object may survive without confirmation.

The theory proves that any false root’s lifetime is bounded—by $\Theta + 1$ epochs under the intrinsic lease, by $\mathcal{L} + 1$ under the extrinsic lease—and that no trace-reachable object is ever reclaimed. The two instantiations are orthogonal: the extrinsic lease requires no header modification and is portable to legacy runtimes; the intrinsic lease eliminates the address-reuse hazard at the cost of one bit. Both are bounded, which shows the mechanism is decoupled from storage layout.

The theory has been validated in a real virtual machine. False roots converge to zero survivors in exactly the predicted number of cycles— $\Theta + 1 = 4$ for transient roots, $P \cdot (\Theta + 1) + 2 = 10$ for oscillating ($P = 2$) roots under DRD. Stop-the-world pauses stay under $25 \mu\text{s}$ regardless of heap size, and no trace-reachable object is ever reclaimed under concurrent marking.

Beyond matching precise GC’s safety in the ideal case, the aging theory exceeds it in the realistic case. Precise GC’s safety is conditional on compiler perfection; a single incorrect stack map causes unbounded use-after-free. The aging theory’s safety is unconditional on compiler precision: it degrades gracefully from 100% (perfect compiler) to bounded retention (imperfect compiler) to bounded retention (no compiler at all), never collapsing to catastrophe. In a world where compiler bugs are not hypothetical—HotSpot, V8, and CLR have all shipped GC crashes from incorrect maps—this graceful degradation is the theory’s most practical contribution.

The hybrid lease (section 4.7) closes the last open boundary: the persistent false root ($P = 1$), which the intrinsic age alone cannot bound. By combining the intrinsic age (efficient for transient and oscillating false roots) with an extrinsic, address-keyed, wall-clock lease (which bounds the persistent case at $\mathcal{L} + 2$), the hybrid configuration gives a finite bound for every false-root category. No category remains at $L = \infty$.

Conservative collection need not be a convenience trade-off. It can have a mathematical safety bound, and that bound can be achieved without compiler cooperation, without precise roots, and without abandoning the simplicity that made conservative collection attractive in the first place.

A Theorem Index

Number	Statement
Theorem 3.1	Under pure color guarding, oscillating false roots survive unboundedly.
Corollary 3.1	Any bound requires severing the renewal channel.
Axiom 4.1	Mark source and mark state must be stored orthogonally.
Axiom 4.2	Conservative-only survival confidence decays monotonically.
Axiom 4.3	Trace-confirmed objects are permanently exempt.
Axiom 4.4	State completeness: scan observes $\text{Stack} \cup \text{Registers}$.
Rules 4.1–4.4	Lease: anchor, no-renew, expire, exempt.
Theorem 4.1	False-root lifetime $\leq \Theta + 1$ (intrinsic) / $\leq \mathcal{L} + 1$ (extrinsic).
Theorem 4.2	Trace-reachable objects are never reclaimed (UAF=0).
Definition 4.7	Demotion-pending state ($\text{Pending}(o) \in \{0, 1\}$).
Rules 4.5–4.8	DRD: defer, confirm, rescue, hold.
Theorem 4.8	DRD safety: reclamation requires two consecutive invisible safepoints.
Theorem 4.9	DRD convergence: transient $\leq \Theta + 3$, oscillating $\leq P(\Theta + 1) + 2$.
Definition 4.10	Hybrid lease: intrinsic age + extrinsic lease table.
Theorem 4.10	Hybrid lease convergence: persistent $\leq \mathcal{L} + 2$ (closes ∞ boundary).
Theorem 7.1 (M1)	Multi-mutator convergence: bound identical to single-mutator.
Theorem 7.2 (M2)	Multi-mutator safety: no trace-reachable object reclaimed.
Theorem 7.3 (M3)	Lease capacity scales as $O(C \cdot T)$, independent of heap.
Theorem 7.4	Graceful degradation: aging theory’s safety ceiling exceeds precise GC under compiler imperfection.

References

- [1] H.-J. Boehm. Space efficient conservative garbage collection. In *PLDI*, 1993.
- [2] H.-J. Boehm. Bounding space usage of conservative garbage collectors. In *POPL*, 2002.
- [3] H. G. Baker. The tethered gc problem. *OOPSLA Addendum*, 1992.
- [4] B. Goldberg. Tag-free garbage collection. PhD thesis, Stanford University, 1991.
- [5] D. Ungar. Generation scavenging: A non-disruptive high performance storage reclamation algorithm. In *SDE*, 1984.